الجمهورية العربية السورية
المعهد العالي للعلوم التطبيقية والتكنولوجيا
قسم النظم المعلوماتية
العام الدراسي 2025/2026

مشروع سنة رابعة

نظام توثيق الشهادات الإلكترونية والتحقق منها بالاعتماد على تقنية سلاسل الكتل والتخزين اللامركزي

تقديم
هلا محمد بركات

إشراف
د. حسين خيّو
ما. مفيد دوزكنجي

6/8/2026

الملخص

لمواجهة مشكلات التلاعب بالسجلات الأكاديمية وبطء التحقق التقليدي المعتمد على قواعد البيانات المركزية، يهدف هذا المشروع إلى تصميم وتنفيذ نظام UniCreds، وهو نظام لامركزي لإصدار الشهادات الجامعية وإلغائها والتحقق منها. يعتمد النظام على العقود الذكية ضمن بيئة إيثيريوم لإدارة دورة حياة الشهادة، وعلى نظام الملفات بين الكواكب IPFS لتخزين ملفات الشهادات خارج السلسلة، بحيث تُحفظ على السلسلة أدلة التحقق فقط، مثل البصمة التشفيرية ومعرّف المحتوى.

تم تطوير عقد ذكي بلغة Solidity يطبق نظام التحكم في الوصول المبني على الأدوار RBAC لتنظيم صلاحيات الإدارة والجهات المصدرة، مع سجل للمؤسسات الأكاديمية المعتمدة وآليات لإصدار الشهادات وإلغائها والتحقق من حالتها ومصدرها. كما تم تطوير تطبيق ويب متعدد البوابات يخدم كلّاً من الإدارة، الجامعة، الطالب، وجهة التحقق. أظهرت اختبارات الوحدة باستخدام Hardhat صحة آليات التفويض، ودقة عمليات الإصدار والإلغاء والتحقق، مما يبيّن قابلية النظام لتقديم نموذج آمن وعملي للتحقق المستقل من الشهادات الأكاديمية.

Abstract

To address the problems of academic record tampering and the slow traditional verification processes that rely on centralized databases, this project aims to design and implement UniCreds, a decentralized system for issuing, revoking, and verifying university certificates. The system relies on smart contracts within the Ethereum environment to manage the certificate lifecycle, and on the InterPlanetary File System (IPFS) to store certificate files off-chain, so that only verification evidence, such as the cryptographic hash and the content identifier, is stored on-chain.

A smart contract was developed in Solidity, applying Role-Based Access Control (RBAC) to regulate the permissions of the administration and issuing entities. The system also includes a registry of accredited academic institutions, along with mechanisms for issuing certificates, revoking them, and verifying their status and source. In addition, a multi-portal web interface was developed to serve the administration, the university, the student, and the verifier. Unit tests using Hardhat demonstrated the correctness of the authorization mechanisms and the accuracy of the issuing, revocation, and verification processes, highlighting the system's potential to provide a secure and practical model for independent academic certificate verification.

المحتويات

الفصل الأول 6

التعريف بالمشروع 6

يتضمن هذا الفصل التعريف بالمشروع ومتطلباته. 6

1.1 مقدمة 6

1.2 هدف المشروع 7

1.3 المتطلبات الوظيفية 7

1.4 المتطلبات غير الوظيفية 9

الفصل الثاني 11

الدراسة التحليلية 11

يوضح هذا الفصل عملية تحليل النظام اللامركزي لتوثيق الشهادات ودراسة متطلباته وتفاعلاته، من خلال تحديد الأطراف المعنية (الفاعلين) ووصف حالات الاستخدام الرئيسية التي تضمن إدارة دورة حياة الاعتماد الأكاديمي بشكل آمن وموثوق. 11

2.1 تحديد الفاعلين (Actors) 11

2.2 مخطط حالات الاستخدام (Use Case Diagram) 12

2.3 السرد النصي لحالات الاستخدام 12

2.3.1 تسجيل مؤسسة أكاديمية وتفويضها 13

2.3.2 إصدار وتوثيق شهادة رقمية 16

2.3.3 إلغاء شهادة صادرة 19

2.3.4 التحقق من صحة ومصدر الشهادة 22

2.3.5 استعراض الطالب لسجله الأكاديمي والتحقق منه..... 24

الفصل الثالث 27

تصميم النظام 27
يَعرض هذا الفصل القرارات التصميمية التي بني من خلالها النظام. 27
3.1 مقدمة 27
3.2 الأهداف والمبادئ التصميمية 27
3.3 المعمارية العامة للنظام 28
3.4 تصميم العقد الذكي (Smart Contract Design) 29
3.4.1 هياكل البيانات الرئيسية 30
3.4.2 نموذج التحكم بالوصول 32
3.4.3 آلية الالتزام بالبصمة (Commitment Hash) 32
3.4.4 التخزين اللامركزي (IPFS / Pinata) 32
3.5 تصميم تدفّقات العمليات 32
3.5.1 تدفّق تسجيل مؤسسة تعليمية ومنح دور الإصدار 32
3.5.2 تدفق إصدار شهادة 33
3.5.3 تدفق إلغاء الشهادة 34
3.5.4 تدفق التحقق من صحة الشهادة 34
3.5.5 تدفق استعراض الطالب لشهاداته 36
3.6 اعتبارات الأمان والخصوصية 37
الفصل الرابع 38
تنجيز النظام والاختبارات ومناقشة النتائج 38
يَعرض هذا الفصل كيفية تنجيز النظام مع تفصيل كل جزء من أجزائه والاختبارات التي تم تطبيقها للتأكد من تلبية النظام للمتطلبات. 38
4.1 مقدمة 38
4.2 تنفيذ العقد الذكي (Smart Contract) 38

4.3 نشر العقد على الشبكة (Deployment) 40
4.4 تنفيذ طبقة التخزين اللامركزي (IPFS / Pinata) 41
4.5 طبقة الربط مع السلسلة في الواجهة 41
4.6 تنفيذ واجهات المستخدم 42
4.7 معالجة الأخطاء وتجربة المستخدم 47
4.8 مطابقة المتطلبات مع التنفيذ 47
4.9 أدوات التنفيذ 50
4.10 بيئة الاختبار وأدواته 50
4.11 اختبار العقد الذكي (Unit Testing) 51
4.12 اختبار التكامل والواجهة 53
4.13 اختبار الأمان (Security Testing) 56
4.13.1 المراجعة اليدوية 56
4.13.2 التحليل الآلي 57
4.14 اختبار الحمل (Load Testing) 57
4.15 تقييم النتائج ومطابقة المتطلبات 58

الفصل الأول

التعريف بالمشروع

يتضمن هذا الفصل التعريف بالمشروع ومتطلباته.

1.1 مقدمة

تُعد الوثائق الأكاديمية والشهادات الجامعية الإثبات الرسمي للمؤهلات العلمية التي يحصل عليها الأفراد، ولذلك فإن سلامة هذه الوثائق وموثوقية طرق التحقق منها تمثلان متطلباً أساسياً للجامعات وجهات التوظيف والمؤسسات التعليمية. إلا أنّ إدارة الشهادات ضمن الأنظمة التقليدية وقواعد البيانات المركزية تواجه عدداً من التحديات الأمنية والتشغيلية، فعلى الرغم من كفاءة هذه الأنظمة وسرعتها داخل البيئات الموثوقة داخلياً، إلا أنها تبقى عرضة لمشكلة "نقطة الفشل الواحدة" (Single Point of Failure)، إذ يمكن أن يؤدي اختراق الخادم المركزي أو إساءة استخدام صلاحيات الإدارة إلى تعديل السجلات أو حذفها أو التلاعب بها. كما أن عمليات التحقق التقليدية غالباً ما تعتمد على المراسلات المباشرة مع الجهة المصدرة، مما يجعلها بطيئة، محدودة الشفافية، وغير مستقلة بالكامل عن المؤسسة المالكة للسجل.

استجابةً لهذه التحديات، تبرز تقنية سلاسل الكتل (Blockchain) بوصفها بنية مناسبة لبناء سجلات رقمية موزعة وقابلة للتحقّق، إذ تستبدل الاعتماد المطلق على الجهة المركزية بآليات تحقق تشفيرية موزعة. وتستند هذه التقنية إلى مفاهيم مثل التجزئة التشفيرية، والتوقيع الرقمي، والربط التتابعي بين الكتل، مما يجعل أي تعديل لاحق في البيانات الموثقة قابلاً للكشف بسبب تغيّر البصمات الرقمية المرتبطة بها.

في سياق هذا المشروع، لا تُخزَّن الوثيقة الأكاديمية الكاملة على سلسلة الكتل، بل تُحفظ عليها أدلة تحقق مختصرة فقط (البصمة التشفيرية ومعرّف المحتوى)، بينما تُخزَّن الوثيقة الأصلية خارج السلسلة عبر نظام الملفات بين الكواكب (IPFS). ويحقق هذا الفصل بين طبقة التخزين وطبقة التوثيق توازناً بين موثوقية التحقق وكفاءة استخدام موارد السلسلة.

بناءً على ذلك، يقدّم مشروع UniCreds نظاماً لامركزياً لإصدار الشهادات الجامعية وإلغائها والتحقق منها، من خلال الجمع بين العقود الذكية على بيئة إيثيريوم، والتخزين اللامركزي عبر IPFS، وآليات التحكم في الوصول المبنية على الأدوار. يهدف النظام إلى تمكين الجهات الأكاديمية المعتمدة من إصدار شهادات رقمية موثّقة، وتمكين الطلاب من ربط شهاداتهم بهويتهم الرقمية، وإتاحة التحقق الفوري لجهات التوظيف أو الجهات الخارجية دون الاعتماد المستمر على قواعد بيانات مركزية.

1.2 هدف المشروع

يهدف هذا المشروع إلى تصميم وتنفيذ نظام لامركزي آمن لإدارة دورة حياة الشهادة الأكاديمية، يعتمد على عقد ذكي مكتوب بلغة Solidity ضمن بيئة متوافقة مع إيثيريوم، وعلى تخزين ملفات الشهادات خارج السلسلة عبر IPFS مع حفظ أدلة التحقق الأساسية فقط داخل العقد.

وتتمثل الأهداف الرئيسية فيما يلي:

1. توثيق دورة حياة الشهادة (الإصدار، الإلغاء، التحقق) على نحو موثوق وقابل للتدقيق.
2. حوكمة الجهات المصدرة عبر سجل للمؤسسات المعتمدة وتنظيم صلاحياتها وفق مبدأ التحكم في الوصول المبني على الأدوار (RBAC).
3. حماية الخصوصية وكفاءة الموارد بالاكتفاء بحفظ البصمة التشفيرية ومعرّف المحتوى على السلسلة وإبقاء الوثيقة على IPFS.
4. إتاحة تحقق مستقل وفوري لأي جهة خارجية دون الرجوع إلى قاعدة بيانات مركزية.
5. توفير واجهة ويب متعددة البوابات تخدم الإدارة والجامعة والطالب وجهة التحقق.

وتُفصَّل هذه الأهداف إجرائياً في المتطلبات الوظيفية التالية.

1.3 المتطلبات الوظيفية

تحدد المتطلبات الوظيفية العمليات الأساسية التي يجب أن يوفرها نظام UniCreds لإدارة دورة حياة الشهادات الأكاديمية والتحقق منها.

يجب أن يوفر النظام ما يلي:

1. **إدارة المؤسسات المعتمدة (Accredited Institutions Management)**: يجب أن يتيح النظام للإدارة تسجيل الجامعات والمؤسسات الأكاديمية المعتمدة مع معلومات تعريفية أساسية (مثل الاسم والموقع) وربطها بعنوان محفظتها.
2. **تفعيل/إيقاف المؤسسات وإدارة حالتها (Institution Status Management)**: يجب أن يدعم النظام تغيير حالة المؤسسة (نشطة/موقوفة) دون حذف سجلها، مع توثيق سبب الإيقاف عند التعليق وإزالته عند إعادة التفعيل.

3. **التحكم في الوصول المبني على الأدوار (Role-Based Access Control – RBAC)**: يجب أن يفرض النظام صلاحيات واضحة تفصل بين الإدارة والجهات المصدرة، وتمنع تنفيذ العمليات الحساسة إلا للحسابات التي تمتلك الدور المناسب.

4. **منح/سحب صلاحية الإصدار تلقائياً حسب حالة المؤسسة (Automatic Issuer Authorization)**: يجب أن تُمنح المؤسسة دور الإصدار عند التسجيل/التفعيل، ويُسحب عنها تلقائياً عند الإيقاف لمنعها من الإصدار أثناء التعليق.

5. **تخزين الشهادات خارج السلسلة عبر IPFS (Off-Chain Certificate Storage via IPFS)**: يجب أن يرفع النظام ملف الشهادة (PDF) إلى IPFS عبر خدمة تثبيت (Pinning) واستخلاص معرّف المحتوى (CID) الخاص بها.

6. **ربط الشهادة بالوثيقة الأصلية (CID Linking)**: يجب أن يربط النظام كل شهادة بـ CID الخاص بملفها الأصلي بحيث يمكن الاستدلال على الوثيقة عند التحقق دون تخزين الملف على السلسلة.

7. **توليد الالتزام/البصمة التشفيرية (Cryptographic Commitment/Hash Generation)**: يجب أن يولد النظام بصمة تشفيرية للشهادة باستخدام Keccak256 بناءً على بيانات الإصدار المعتمدة (هوية المصدر، الطالب، البيانات الأكاديمية، CID، وعدّاد الإصدار) لضمان النزاهة.

8. **إصدار الشهادات (Certificate Issuance)**: يجب أن يسمح النظام للمؤسسات المعتمدة والنشطة بإصدار شهادة جديدة وتسجيل بياناتها الأساسية على السلسلة (الحالة، تاريخ الإصدار، المصدر، الطالب، CID) مع منع تكرار السجل.

9. **إلغاء الشهادات من المصدر الأصلي فقط (Issuer-Only Revocation)**: يجب أن يتيح النظام إلغاء الشهادة فقط للجهة التي أصدرتها أول مرة، مع تحويل حالتها إلى "ملغاة" دون حذفها للحفاظ على الأثر التاريخي.

10. **حفظ سبب الإلغاء (Revocation Reason Tracking)**: يجب أن يدعم النظام تخزين سبب إلغاء الشهادة كسجل نصي مرتبط بها لتقديم تفسير إداري/قانوني عند التحقق.

11. **التحقق العام من الشهادات (Public Certificate Verification)**: يجب أن يتيح النظام لأي جهة التحقق من وجود الشهادة وحالتها ومصدرها وتاريخها وCID الخاص بها عبر توابع قراءة عامة (View).

12. **توثيق العمليات عبر الأحداث (Event Logging / Audit Trail)**: يجب أن يطلق النظام أحداثاً عند تسجيل جامعة، تغيير حالتها، إصدار شهادة، وإلغاء شهادة، لتمكين التتبع وبناء واجهات شفافة.

13. **واجهة ويب متعددة البوابات حسب الدور (Multi-Portal Web Interface)**: يجب أن يوفر النظام واجهات منفصلة تخدم: مدير النظام لإدارة الجامعات، الجامعة للإصدار والإلغاء، الطالب للاستعراض، وجهة التحقق للتحقق.

1.4 المتطلبات غير الوظيفية

إلى جانب الوظائف الأساسية، يجب أن يتمتع النظام بمجموعة من الخصائص التقنية والقيود الهندسية التي تضمن كفاءته واستدامته في بيئات الإنتاج الفعلية، وتتضمن:

1. **الأمان (Security)**: يجب أن يمنع النظام الوصول غير المصرح به إلى وظائف الإصدار والإلغاء وإدارة المؤسسات، وأن يضمن تنفيذ العمليات الحساسة من قبل الجهات المخولة فقط.
2. **الخصوصية (Privacy)**: يجب ألّا تُخزن البيانات الأكاديمية الحساسة أو ملفات الشهادات الأصلية بشكل مباشر على سلسلة الكتل، بل تُحفظ فقط أدلة تحقق مختصرة تدعم التحقق دون كشف تفاصيل غير ضرورية.
3. **نزاهة البيانات وعدم القابلية للتلاعب (Data Integrity & Immutability)**: يجب أن يضمن النظام أن سجلات الشهادات لا يمكن تعديلها أو تزويرها بعد توثيقها، فأي تغيير في بيانات الشهادة أو ملفها الأصلي يؤدي إلى اختلاف البصمة التشفيرية أو الـ CID، مما يكشف التلاعب فور التحقق.
4. **كفاءة التخزين واستهلاك الموارد (Storage and Resource Efficiency)**: يجب أن يقلل النظام حجم البيانات المخزنة داخل العقد الذكي، من خلال الاعتماد على البصمات التشفيرية ومعرّفات المحتوى بدلاً من تخزين الملفات الكاملة على السلسلة.
5. **قابلية التوسع (Scalability)**: يجب أن تكون بنية النظام مناسبة للتعامل مع عدد كبير من الشهادات مستقبلاً، وذلك عبر فصل تخزين الملفات عن منطق التوثيق، وتقليل العمليات المكلفة داخل سلسلة الكتل.
6. **الموثوقية (Reliability)**: قدرة النظام على تنفيذ وظائفه الأساسية (التسجيل، الإصدار، الإلغاء، التحقق) بشكل صحيح ومتسق دون أخطاء تؤثر على سلامة السجلات أو نتائج التحقق.
7. **التوافرية والاستمرارية (Availability & Continuity)**: قدرة المستخدمين وجهات التحقق على الوصول إلى خدمات النظام وقراءة بيانات التحقق عند الحاجة دون الاعتماد على نقطة مركزية واحدة، مع بقاء الشهادات قابلة للاستعلام والتحقق على المدى الطويل حتى عند تغيّر حالة الجهة المصدرة (تفعيل/إيقاف) أو تغيّر ظروف التشغيل.
8. **قابلية الاستخدام (Usability)**: يجب أن تكون واجهات النظام واضحة وبسيطة لكل طرف (مدير النظام، الجامعة، الطالب، جهة التحقق)، بحيث يمكن تنفيذ المهام الأساسية (تسجيل/إصدار/إلغاء/تحقق) بسهولة دون الحاجة لمعرفة تقنية عميقة بسلاسل الكتل.

9. **قابلية الصـــيانة (Maintainability)**: يجب أن يكون النظام قابلاً للتعديل والتطوير لاحقاً دون كســـر الوظائف الأساسية، من خلال فصل المكوّنات (عقد ذكي، سكربتات، واجهة) واعتماد بنية وتنظيم كود يسهل فهمه وتحديثه.
10. **قابلية الاختبار (Testability)**: يجب أن يدعم النظام اختبارات آلية تغطي الوظائف الجوهرية (التســـجيل، التفويض، الإصدار، الإلغاء، التحقق) لضمان اكتشاف الأخطاء مبكراً والتحقق من صحة السلوك قبل أي نشر.
11. **التوافقية وقابلية التكامل (Interoperability & Integrability)**: يجب أن يتيح النظام التكامل مع مكونات وخدمات خارجية مثل محافظ Web3، بوابات IPFS/Pinning، عبر واجهات ومعايير شـائعة تسـهّل الربط دون تغييرات جذرية.

الفصل الثاني

الدراسة التحليلية

يوضح هذا الفصل عملية تحليل النظام اللامركزي لتوثيق الشهادات ودراسة متطلباته وتفاعلاته، من خلال تحديد الأطراف المعنيّة (الفاعلين) ووصف حالات الاستخدام الرئيسية التي تضمن إدارة دورة حياة الاعتماد الأكاديمي بشكل آمن وموثوق.

2.1 تحديد الفاعلين (Actors)

يتفاعل مع النظام اللامركزي أربعة أطراف رئيسية، يمتلك كل منها دوراً برمجياً محدداً وصلاحيات مقيدة تضمن مبدأ "فصل المهام" (Separation of Duties):

- **مدير النظام (System Administrator)**

يمثل السلطة التنظيمية العليا والجهة المشرفة التي قامت بنشر العقد الذكي (Deployer). يمتلك هذا الفاعل الدور التشفيري (DEFAULT_ADMIN_ROLE). تقتصر مهامه على الرقابة والحوكمة، حيث يقوم بتسجيل الجامعات المعتمدة، وإدارة حالاتها (تفعيل أو تعليق)، دون أن يمتلك صلاحية إصدار الشهادات مباشرة.

- **المؤسسة المعتمدة / المصدر (Authorized Issuer)**

تمثل الهيئات التعليمية (الجامعات) التي تم التحقق من هويتها من قبل الإدارة العليا. تمتلك هذه الجهة دور المصدر (ISSUER_ROLE)، وتتركز مسؤوليتها الحصرية في توثيق الشهادات الأكاديمية للطلاب وإصدارها على سلسلة الكتل، بالإضافة إلى صلاحية إبطال الشهادات التي قامت هي بإصدارها مسبقاً.

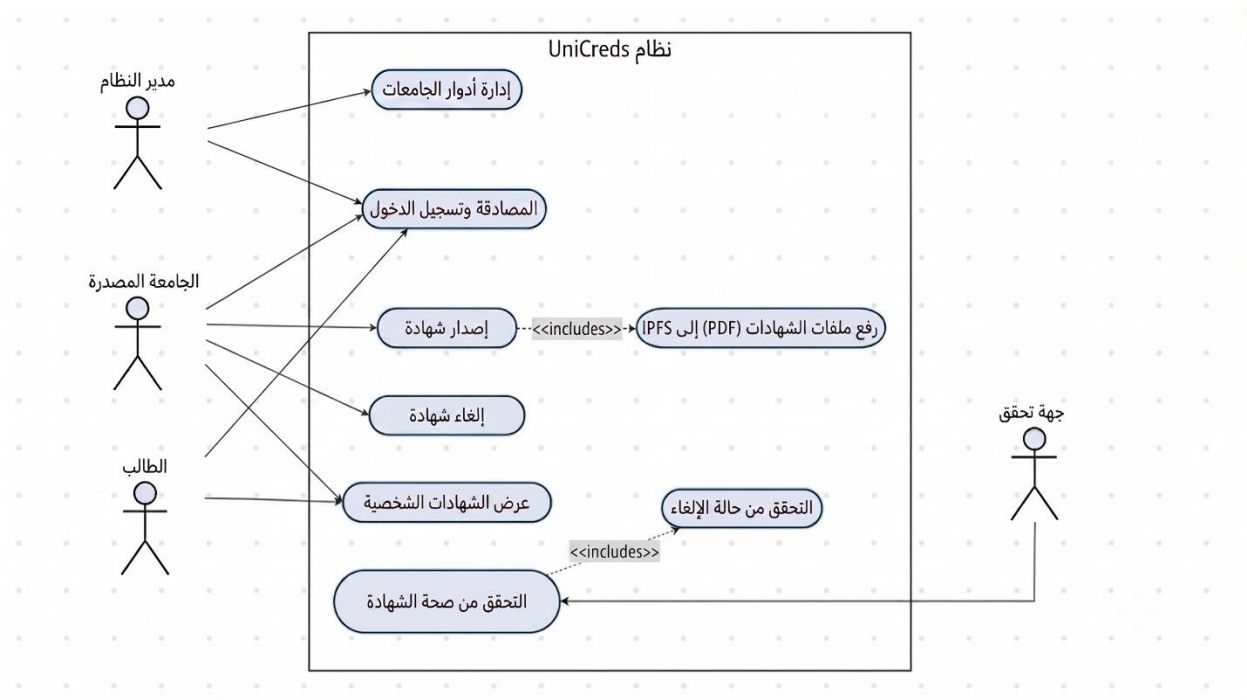
- **الطالب (Student)**

هو المستفيد النهائي والمالك الشرعي للاعتماد الأكاديمي. يُعرّف الطالب داخل النظام من خلال عنوان محفظته الرقمية (Wallet Address) الذي يتم ربط الشهادة به بشكل غير قابل للفصل، مما يمنحه السيادة الكاملة على هويته الرقمية.

- **جهة التحقق (Verifier)**

تمثل أي جهة خارجية (مثل أصحاب العمل، أو جامعات أخرى، أو تطبيقات لامركزية dApps) ترغب في التأكد من موثوقية الشهادة. يتفاعل هذا الكيان مع النظام من خلال توابع القراءة العامة (View Functions) التي لا تستهلك أي رسوم (Gas)، لاستخراج حالة الشهادة وهوية الجامعة المصدرة لها.

2.2 مخطط حالات الاستخدام (Use Case Diagram)



صورة 1 – مخطط حالات الاستخدام لنظام **UniCreds**

2.3 السرد النصي لحالات الاستخدام

يوضح هذا القسم السيناريوهات التشغيلية الجوهرية للنظام من خلال جداول تفصيلية لحالات الاستخدام.

2.3.1 تسجيل مؤسسة أكاديمية وتفويضها

الجدول 1 – تسجيل مؤسسة أكاديمية وتفويضها

اسم الحالة: تسجيل مؤسسة أكاديمية وتفويضها	
الوصف Description	قيام الإدارة العليا بتسجيل كيان تعليمي جديد في سجل المؤسسات ومنحه الصلاحيات التشفيرية اللازمة للعمل.
الفاعلين Actors	مدير النظام (يبدأ العملية وينفّذها)، النظام (يعالج الطلب على السلسلة).
الشروط السابقة Precondition	– المستدعي يمتلك صلاحية الإدارة العليا (DEFAULT_ADMIN_ROLE). – عنوان المؤسسة المراد تسجيلها غير مسجل مسبقاً في السجل.
الشروط اللاحقة Postcondition	– إنشاء ملف تعريفي للمؤسسة (الاسم والموقع) وتوثيق الختم الزمني للتسجيل. – تفعيل حالة المؤسسة لتصبح نشطة (isActive = true). – منح المؤسسة دور الإصدار التنفيذي (ISSUER_ROLE). – انبعاث حدث الإشعار (UniversityRegistered).

سير الأحداث

السيناريو الأساسي الناجح

الجدول 2 – السيناريو الناجح لحالة استخدام تسجيل مؤسسة أكاديمية وتفويضها

مدير النظام	النظام (العقد الذكي على سلسلة الكتل)
1. يستدعي التابع registerUniversity ممرراً عنوان المؤسسة الرقمي، اسمها، وموقعها الجغرافي.	

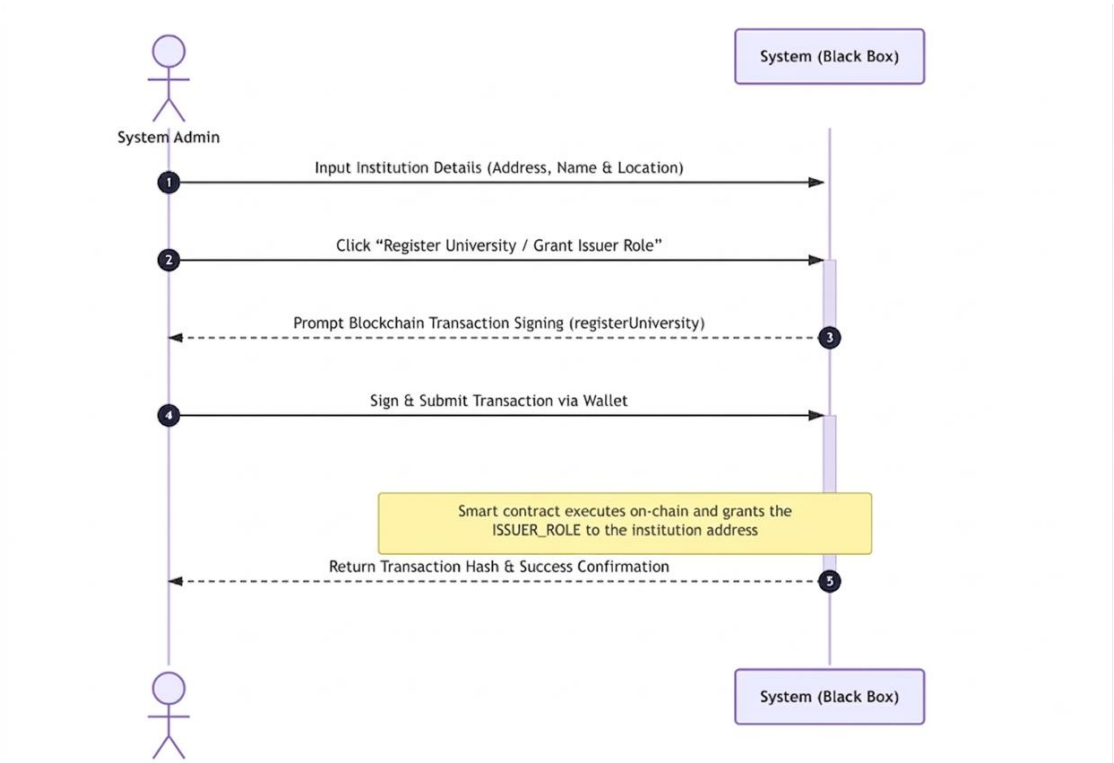
	2. يتحقق النظام من امتلاك المستدعي لدور الإدارة العليا (DEFAULT_ADMIN_ROLE).
	3. يتأكد النظام من صحة العنوان وأن المؤسسة غير مسجلة مسبقاً.
	4. يقوم بإنشاء الملف التعريفي للمؤسسة (UniversityProfile) ويفعّل حالة نشاطها (isActive = true) مع توثيق الختم الزمني.
	5. يمنح النظام آلياً صلاحية الإصدار التنفيذية (ISSUER_ROLE) لعنوان المؤسسة لتتزامن مع حالتها الإدارية.
	6. يُطلق الحدث البرمجي UniversityRegistered للإشعار بنجاح العملية.

وتوضّح الصورة (2) التالية هذا السيناريو بتمثيل تسلسلي لتفاعل مدير النظام مع النظام.

المسارات البديلة

A1: بدلاً من تسجيل مؤسسة جديدة بالكامل، يمكن لمدير النظام (في المرحلة 1) استدعاء التابع toggleUniversityStatus للتعامل مع مؤسسة مسجلة مسبقاً كانت في حالة "إيقاف". يؤدي هذا المسار البديل إلى عكس حالتها لتصبح نشطة مجدداً (isActive = true)، ويقوم النظام آلياً بمنحها دور الإصدار (ISSUER_ROLE) مرة أخرى، مع إطلاق حدث الإشعار UniversityStatusToggled بدلاً من حدث التسجيل الجديد.

ويمكن لمدير النظام استخدام التابع نفسه toggleUniversityStatus لتعليق مؤسسة نشطة، ويشترط النظام تمرير سبب تعليق غير فارغ، وإلا رُفضت المعاملة بخطأ ("Suspension reason required").



صورة 2 – مخطط تسلسل النظام – تسجيل مؤسسة أكاديمية وتفويضها

مسارات الأخطاء

E1: في المرحلة 1، إدخال عنوان مؤسسة غير صالح (العنوان الصفري (address(0)).

يرفض النظام المعاملة ويُرجع خطأ ("Invalid address") قبل إنشاء أي سجل.

E2: في المرحلة 2، فشل التحقق من الصلاحيات (المستدعي لا يملك دور الإدارة العليا DEFAULT_ADMIN_ROLE).

يقوم النظام برفض المعاملة فوراً، وإيقاف التنفيذ، وإرجاع خطأ مخصص (AccessControlUnauthorizedAccount).

E3: في المرحلة 3، فشل التحقق من حالة التسجيل (عنوان المؤسسة المدخل مسجل مسبقاً في قاعدة البيانات).

يقوم النظام بالتراجع عن المعاملة (Revert) وإرجاع رسالة خطأ صريحة ("Already registered") لمنع تداخل السجلات.

2.3.2 إصدار وتوثيق شهادة رقمية

الجدول 3 – إصدار وتوثيق شهادة رقمية

اسم الحالة: إصدار وتوثيق شهادة رقمية	
الوصف Description	تسجيل بصمة السجل الأكاديمي للطالب وربطها بالهوية المؤسساتية وموقع التخزين اللامركزي.
الفاعلين Actors	المؤسسة المعتمدة (الجامعة)، النظام.
الشروط السابقة Precondition	– حساب المؤسسة يمتلك دور (ISSUER_ROLE). – حساب المؤسسة نشط إدارياً داخل السجل (isActive == true). – بصمة الشهادة (Hash) غير مكررة ولم يتم توثيقها مسبقاً. – تم رفع ملف الوثيقة الأصلي إلى IPFS واستخراج معرف المحتوى (CID) الخاص به.
الشروط اللاحقة Postcondition	– تخزين البيانات المهيكلة (عنوان الطالب، البصمة الرقمية، ipfsCID، هوية المصدر، والختم الزمني) في سلسلة الكتل. – تعيين حالة الشهادة كصالحة (isValid = true). – انبعاث حدث الإصدار (CertificateIssued).

سير الأحداث

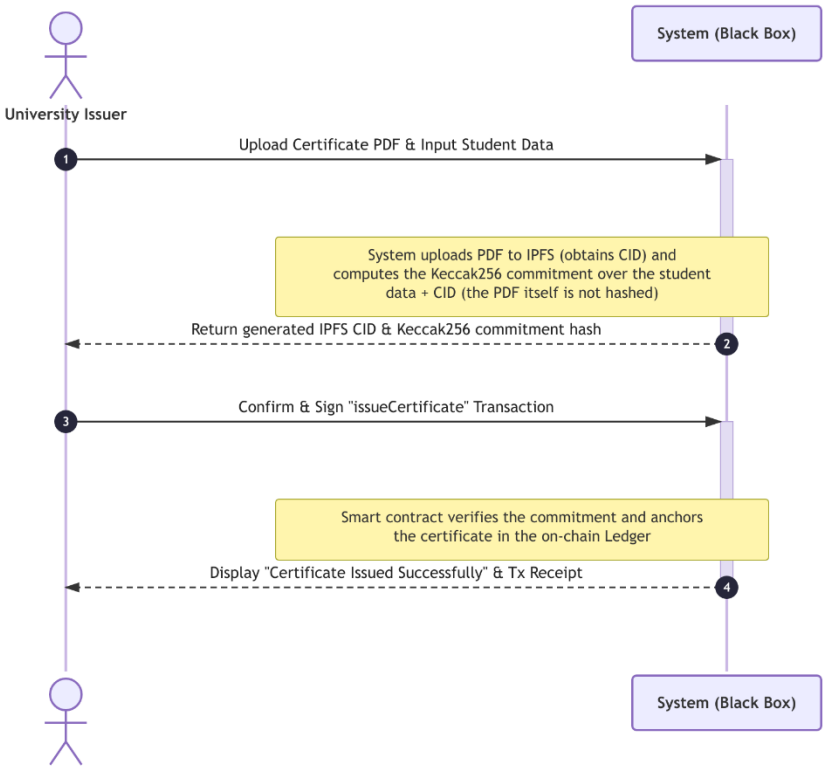
السيناريو الأساسي الناجح

الجدول 4 – السيناريو الناجح لحالة استخدام إصدار وتوثيق شهادة رقمية

الفاعل (المؤسسة المعتمدة)	النظام (IPFS + العقد الذكي)
1. تُشغّل المؤسسة سكربت الإصدار المتكامل (issueCredential.ts) وتُدخل ملف الـ PDF للشهادة وبيانات الطالب الوصفية.	
	2. يتصل النظام ببوابة Pinata، ويرفع الملف إلى شبكة IPFS، ثم يستخلص معرف المحتوى الفريد (CID).
	3. يتصل النظام بالعقد الذكي لاستدعاء تابع generateCertificateHash وتوليد البصمة الرقمية للبيانات.
4. توقع المؤسسة معاملة الاستدعاء للتابع issueCertificate ممرّرةً البصمة الرقمية، وعنوان الطالب، ومعرّف المحتوى (CID)، بالإضافة إلى البيانات الوصفية (رقم الطالب، التخصص، المعدّل) اللازمة لإعادة توليد البصمة والتحقق منها.	
	5. يتحقق العقد من امتلاك المؤسسة لدور الإصدار (ISSUER_ROLE) وأنها نشطة إدارياً في سجل المؤسسات.
	6. يعيد العقد حساب البصمة المتوقّعة (expectedHash) داخلياً من المدخلات نفسها، ويقارنها بالبصمة المُمرَّرة للتأكد من سلامة البيانات وعدم التلاعب بها.

	7. يتأكد العقد من عدم تكرار البصـــمة الرقمية لحماية النظام.
	8. يقوم بحفظ البيانات المهيكلة للشـهادة بشـكل دائم في مصـفوفة السـجلات (certificates) ويربطها بعنوان المؤسسة المصدرة (msg.sender).
	9. يُطلق الحدث البرمجي CertificateIssued للإشـعار بنجاح التوثيق.

وتُجسّد الصورة التالية التدفق التسلسلي لعملية الإصدار بطبقتيها (خارج السلسلة وداخلها).



صورة 3 – مخطط تسلسل النظام – إصدار وتوثيق شهادة رقمية

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: في المرحلة 2، فشل الاتصال بشبكة التخزين اللامركزي أو فشل المصادقة (انقطاع الاتصال مع بوابة Pinata، أو مفتاح وصول JWT غير صالح/مفقود).

يقوم سكربت الأتمتة بالتقاط الخطأ (عبر كتلة try...catch)، وتسجيله في واجهة المطور، وإنهاء المهمة بفشل قبل إرسال أي معاملة إلى سلسلة الكتل لتوفير رسوم الغاز.

E2: في المرحلة 5، فشل التحقق من الصلاحية التنفيذية (المؤسسة موقوفة إدارياً داخل السجل أو لا تملك الدور ISSUER_ROLE).

يقوم العقد الذكي برفض المعاملة فوراً وإرجاع رسالة خطأ ("Issuer is suspended in registry" أو خطأ الصلاحيات) لحماية النظام.

E3: في المرحلة 6، اكتشاف تكرار في البيانات التشفيرية (البصمة الرقمية للشهادة Hash تم توثيقها مسبقاً).

يقوم النظام برفض العملية وإرجاع رسالة خطأ ("Certificate hash already exists") لمنع هجمات إعادة الإصدار وحماية تفرد الوثائق.

E4: في المرحلة 6، فشل تطابق البصمة (البصمة المُمرَّرة لا تطابق البصمة المعاد حسابها من المدخلات).

يقوم العقد بالتراجع عن المعاملة (Revert) وإرجاع خطأ صريح ("Invalid certificate hash") لضمان عدم قبول بيانات غير متسقة.

2.3.3 إلغاء شهادة صادرة

الجدول 5 – حالة استخدام إلغاء شهادة صادرة

اسم الحالة: إبطال شهادة صادرة	
الوصف Description	قيام الجامعة بتغيير الصفة القانونية الرقمية لشهادة أصدرتها سابقاً إلى حالة "ملغاة" لأسباب أكاديمية، دون حذف أثرها التاريخي.
الفاعلين Actors	المؤسسة المعتمدة (الجامعة المصدرة حصراً)، النظام.
الشروط السابقة Precondition	– حساب المؤسسة المستدعية يمتلك دور الإصدار (ISSUER_ROLE). – الشهادة مسجلة وموجودة فعلياً على السلسلة.

	– الشهادة في حالة صالحة حالياً. – عنوان المحفظة المستدعية يطابق عنوان المؤسسة التي أصدرت الشهادة أول مرة.
الشروط اللاحقة Postcondition	– تغيير حالة الصلاحية برمجياً إلى غير صالحة (isValid = false). – حفظ سبب الإلغاء نصياً في سجل الأسباب (revocationReasons) للحفاظ على الأثر التاريخي والشفافية. – إطلاق حدث الإلغاء للتنبيه الخارجي (CertificateRevoked).

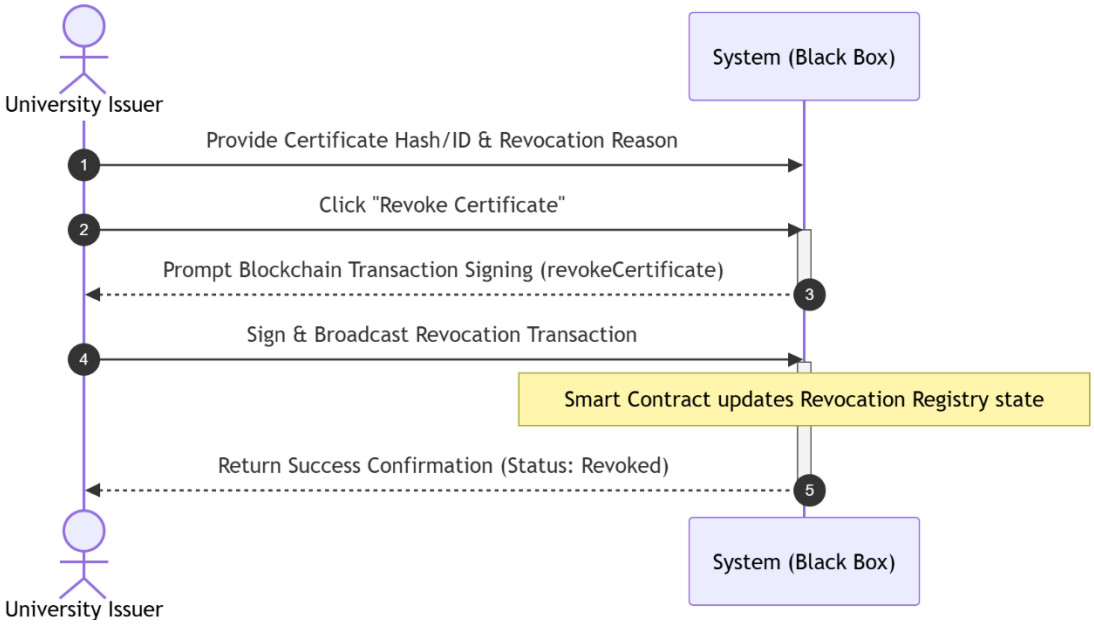
سير الأحداث

السيناريو الأساسي الناجح

الجدول 6 – السيناريو الناجح لحالة استخدام إلغاء شهادة صادرة

الفاعل (المؤسسة المعتمدة)	النظام (العقد الذكي على سلسلة الكتل)
1. تستدعي المؤسسة تابع الإلغاء revokeCertificate وتمرر البصمة الرقمية للشهادة المراد إلغاءها مرفقةً بسبب الإلغاء.	
	2. يتحقق النظام من أن عنوان المستدعي يطابق عنوان الجامعة المصدرة الأصلية للشهادة المحفوظة في السجل.
	3. يتأكد النظام من وجود الشهادة فعلياً (issuDate ≠ 0) ومن أنها ما تزال صالحة (لم تُلغَ مسبقاً).
	4. يغير النظام الحالة المنطقية للشهادة إلى غير صالحة (isValid = false) ويحفظ سبب الإلغاء في السجل (revocationReason).
	5. يُطلق الحدث البرمجي CertificateRevoked لتنبيه الواجهات الخارجية بإلغاء الوثيقة.

وتبيّن الصورة التالية التمثيل التسلسلي لعملية الإلغاء.



صورة 4 – مخطط تسلسل النظام – إلغاء شهادة صادرة

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: في المرحلة 2، فشل مطابقة الهوية المرجعية للمصدر (عنوان المحفظة المستدعية لا يطابق عنوان الجامعة التي أصدرت الشهادة أول مرة).

يقوم النظام برفض المعاملة فوراً وإرجاع رسالة خطأ ("Not original issuer") لحماية حقوق المصدر الأصلي. (ملاحظة: هذا الفحص يسبق فحص الوجود في العقد، لذا تُرفض الشهادات غير الموجودة أيضاً برسالة "Not original issuer" لأن مُصدِرها المخزَّن هو العنوان الصفري).

E2: في المرحلة 3، فشل التحقق من قابلية الإلغاء (الشهادة غير موجودة أصلاً في السجل، أو أنها مُلغاة مسبقاً).

يقوم النظام بالتراجع عن المعاملة وإرجاع خطأ ("Certificate does not exist" أو"Certificate is already revoked").

E3: في مرحلة التحقق من الصلاحية، فقدان المستدعي لدور الإصدار (ISSUER_ROLE) —كأن تكون المؤسسة قد عُلِّقت إدارياً وسُحب منها الدور.

يرفض العقد المعاملة ويُرجع خطأ الصلاحيات (AccessControlUnauthorizedAccount)، وهو أثر مقصود لسياسة الحوكمة يمنع المؤسسة المعلِّقة من إلغاء شهاداتها.

2.3.4 التحقق من صحة ومصدر الشهادة

الجدول 7 – التحقق من صحة ومصدر الشهادة

اسم الحالة: التحقق من صحة ومصدر الشهادة	
الوصف Description	استعلام طرف خارجي عن حالة وثيقة أكاديمية باستخدام بصمتها الرقمية للتأكد من نزاهتها ومصدرها.
الفاعلين Actors	جهة التحقق، النظام.
الشروط السابقة Precondition	امتلاك جهة التحقق للبصمة الرقمية للشهادة المراد الاستعلام عنها.
الشروط اللاحقة Postcondition	إرجاع مصفوفة بيانات دقيقة تشمل (حالة الصلاحية، عنوان الطالب، الختم الزمني للإصدار، عنوان الجامعة المصدِّرة، ومعرّف المحتوى على IPFS وهو ipfsCID).

سير الأحداث

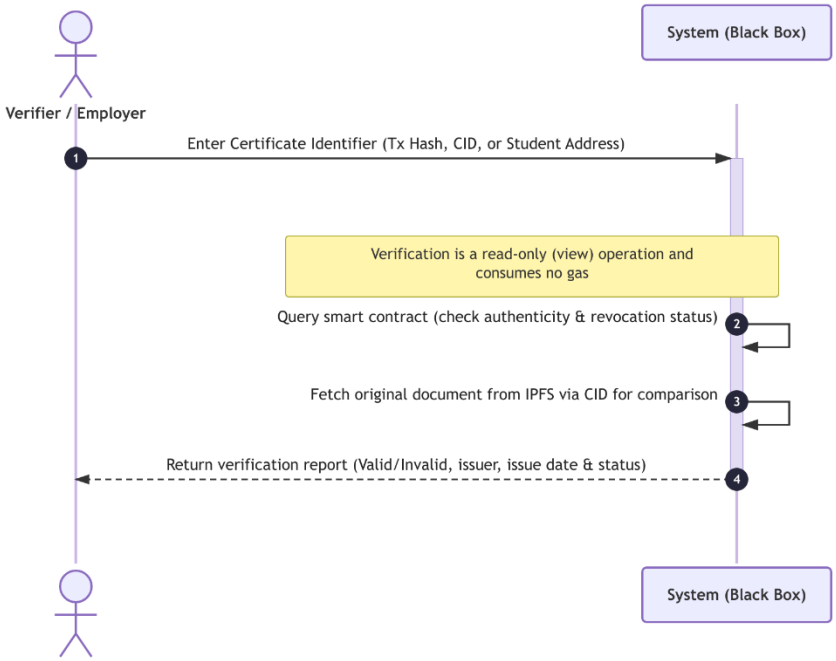
السيناريو الأساسي الناجح

الجدول 8 – السيناريو الناجح لحالة استخدام التحقق من صحة ومصدر الشهادة

الفاعل (جهة التحقق)	**النظام (العقد الذكي + بوابة IPFS)**
1. تُرسل جهة التحقق طلباً استعلامياً لتابع verifyCertificate ممررةً البصمة الرقمية للشهادة.	
	2. يبحث العقد الذكي في مصفوفة السجلات عن البيانات المرتبطة بالبصمة.

	3. يتأكد النظام من وجود السجل عبر فحص الختم الزمني (issueDate != 0).
	4. يُرجع النظام مصفوفة القيم التي تشمل: حالة الصلاحية، عنوان الطالب، تاريخ الإصدار، عنوان الجامعة المصدِّرة، ومعرّف المحتوى (ipfsCID) اللازم لجلب الوثيقة الأصلية.
5. تستخدم جهة التحقق الـ CID المحفوظ لاستدعاء الملف الأصلي.	
	6. تجلب شبكة IPFS الوثيقة الأصلية وتعرضها للمحقق لمطابقتها والتأكد التام.

وتوضّح الصورة التالية مسار التحقق بوصفه استدعاء قراءة عاماً ومجانياً.



صورة 5 – مخطط تسلسل النظام – التحقق من صحة ومصدر الشهادة

المسارات البديلة

A1: إذا كانت الشهادة موجودة لكنها مُلغاة، يُكمل العقد الاستعلام ويُرجع بياناتها كاملة مع حالة صلاحية غير صالحة (isValid = false)، مما يتيح لجهة التحقق معرفة أن الوثيقة كانت موثّقة ثم سُحب اعتمادها.

مسارات الأخطاء

E1: في المرحلة 3، فشل العثور على البصمة الرقمية داخل العقد الذكي (الشهادة مزورة ولم يتم توثيقها مسبقاً على الشبكة).

يقوم النظام بإنهاء عملية الاستعلام وإرجاع نتيجة سلبية (False) مقترنة بقيم صفرية، مما يثبت للمحقق عدم صحة الوثيقة.

2.3.5 استعراض الطالب لسجله الأكاديمي والتحقق منه

الجدول 9 – استعراض الطالب لسجله الأكاديمي والتحقق منه

اسم الحالة: استعراض الطالب لسجله الأكاديمي والتحقق منه	
الوصف Description	قيام الطالب بالدخول إلى النظام باستخدام محفظته الرقمية لاستعراض تفاصيل شهادته، والتأكد من دقة البيانات المخزنة على السلسلة، ومطابقة ملف الوثيقة الأصلي المرفوع على شبكة التخزين اللامركزي قبل مشاركته مع أطراف خارجية.
الفاعلين Actors	الطالب، النظام.
الشروط السابقة Precondition	– الجامعة المعتمدة قامت بإصدار وتوثيق الشهادة فعلياً على النظام. – الطالب يمتلك البصمة الرقمية (Hash) الخاصة بشهادته أو متصل بمحفظته الرقمية.
الشروط اللاحقة Postcondition	استرجاع بيانات الشهادة الموثقة وعرض الوثيقة الأصلية (ملف PDF) للطالب ليتمكن من مراجعتها.

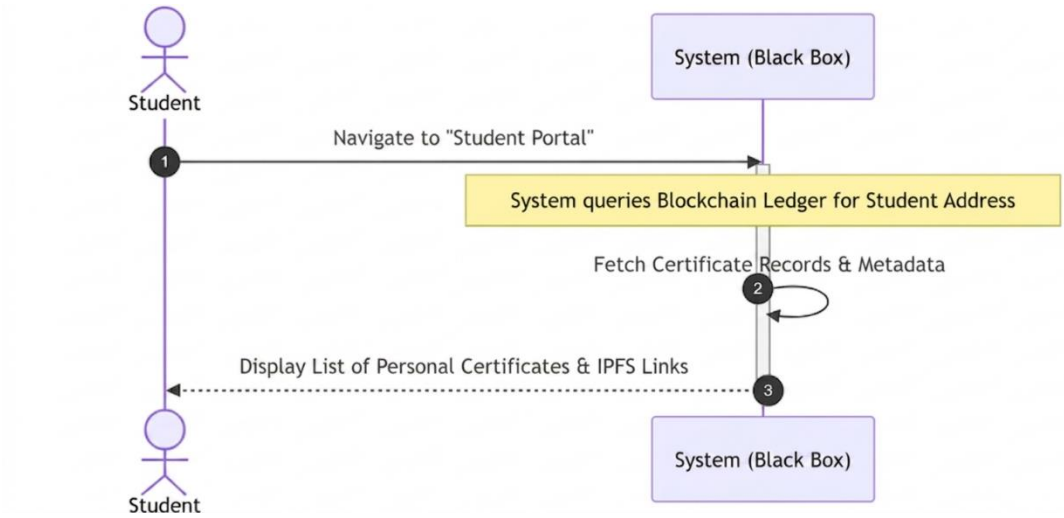
سير الأحداث

السيناريو الأساسي الناجح

الجدول 10 – السيناريو الناجح لحالة استخدام استعراض الطالب لسجله الأكاديمي والتحقق منه

الفاعل (الطالب)	النظام (العقد الذكي + بوابة IPFS)
1. يُرسل الطالب طلباً استعلامياً لتابع verifyCertificate ممرراً البصمة الرقمية لشهادته.	
	2. يبحث العقد الذكي في مصفوفة السجلات عن البيانات المرتبطة بالبصمة.
	3. يتأكد العقد من وجود السجل (issueDate ≠ 0) ويُرجع بياناته، ومنها عنوان الطالب المالك.
	4. يُرجع النظام مصفوفة القيم (حالة الصلاحية، تاريخ الإصدار، وعنوان الجامعة المصدرة).
5. تقارن الواجهة الأمامية عنوان محفظة الطالب المتصل مع عنوان الطالب المعاد من التابع، فإن تطابقا عرضت الواجهة السجل بوصفه ملكاً له.	
6. يستدعي الطالب (عبر الواجهة) الوثيقة الأصلية باستخدام معرف المحتوى (CID).	
	7. تقوم بوابة IPFS بجلب ملف الشهادة الأصلي وعرضه للطالب للتأكد من خلوه من الأخطاء.

وتُظهر الصورة (6) كيفية استرجاع الطالب لسجله اعتماداً على عنوان محفظته.



صورة 6 – مخطط تسلسل النظام – استعراض الطالب لسجله الأكاديمي والتحقق منه

المسارات البديلة

A1: في المرحلة 3، إذا وجد النظام أن الشهادة موجودة لكن تم إلغاؤها من قبل الجامعة (لسبب أكاديمي أو خطأ في الإصدار المسبق)، يُكمل النظام الاستعلام ويُرجع البيانات كاملة ولكن مع تعيين حالة الصلاحية إلى غير صالحة (isValid = false). هذا المسار يتيح للطالب معرفة أن جامعته قد سحبت الاعتماد من هذه النسخة المحددة.

مسارات الأخطاء

E1: في المرحلة 3، فشل العثور على الشهادة (الجامعة لم تقم بإصدار وتوثيق الشهادة بعد، أو أن البصمة المدخلة خاطئة).

يقوم النظام بإنهاء عملية الاستعلام وإرجاع نتيجة سلبية مقترنة بقيم صفرية، مما ينبه الطالب لمراجعة الجامعة المصدرة.

E2: في المرحلة 6، فشل استدعاء الوثيقة من ال IPFS (الـ CID غير صحيح، أو الملف لم يعد مثبّتاً Pinned على شبكة IPFS).

تقوم بوابة العرض (Gateway) بإرجاع خطأ (Timeout أو Not Found 404)، ولن تتمكن جهة التحقق من عرض أو تحميل ملف الـ PDF الخاص بالشهادة.

الفصل الثالث

تصميم النظام

يَعرض هذا الفصل القرارات التصميمية التي بني من خلالها النظام.

3.1 مقدمة

يتناول هذا الفصل الجانب التصميمي لنظام توثيق الشهادات الأكاديمية والتحقق منها المبني على تقنية سلسلة الكتل (Blockchain)، وذلك انتقالاً من متطلبات النظام التي عُرضت في الفصل السابق إلى بنيته المعمارية وتفاصيل مكوّناته وتدفّقات عملياته. يهدف الفصل إلى تقديم صورة هندسية متكاملة توضّح كيفية توزيع المسؤوليات بين طبقات النظام، وكيف تتفاعل هذه الطبقات فيما بينها لتحقيق الأهداف الأساسية المتمثّلة في ضمان سلامة الشهادات، ومنع التزوير، والتحقق اللامركزي دون الحاجة إلى وسيط موثوق مركزي، مع الحفاظ على خصوصية بيانات الطلبة.

ويُبنى العرض على مبدأ الفصل بين المخطط المعماري عالي المستوى الذي يوضّح المكوّنات وحدود الثقة، ومخططات التدفّق التفصيلية التي تصف خطوات كل عملية على حدة.

3.2 الأهداف والمبادئ التصميمية

استند تصميم النظام إلى مجموعة من المبادئ الموجِّهة:

1. **اللامركزية وانعدام الوسيط الموثوق:** إسناد مصدر الحقيقة (Source of Truth) إلى العقد الذكي على شبكة الإيثيريوم بدلاً من قاعدة بيانات مركزية، بحيث يستطيع أي طرف التحقق من الشهادة دون الرجوع إلى الجهة المصدِرة.
2. **سلامة البيانات وعدم قابلية التلاعب (Integrity & Immutability):** تخزين بصمة تشفيرية (Commitment Hash) للشهادة على السلسلة، بحيث يؤدي أي تغيير في محتوى الوثيقة إلى اختلاف البصمة وكشف التزوير فوراً.
3. **خصوصية البيانات (Privacy by Design):** عدم تخزين البيانات الشخصية الحسّاسة (المعدّل، الاختصاص، الرقم الجامعي) على السلسلة العامة، والاكتفاء بتخزين البصمة ومعرّف المحتوى (CID) وعنوان الطالب.

4. **كفاءة التكلفة (Gas Efficiency)**: إبقاء عمليات القراءة والتحقق مجانية تماماً خارج السلسلة، وقصر المعاملات المكلِّفة (Gas) على عمليات تغيير الحالة الضرورية فقط.

5. **مبدأ الامتياز الأدنى (Least Privilege)**: فصل الصلاحيات عبر نظام أدوار صارم، بحيث لا يملك أي طرف صلاحيات تتجاوز وظيفته.

6. **الفصل بين مناطق الثقة (Trust Boundaries)**: عزل البيئة الموثوقة على السلسلة عن بيئة العميل عن البيئة الخارجية غير الموثوقة.

3.3 المعمارية العامة للنظام

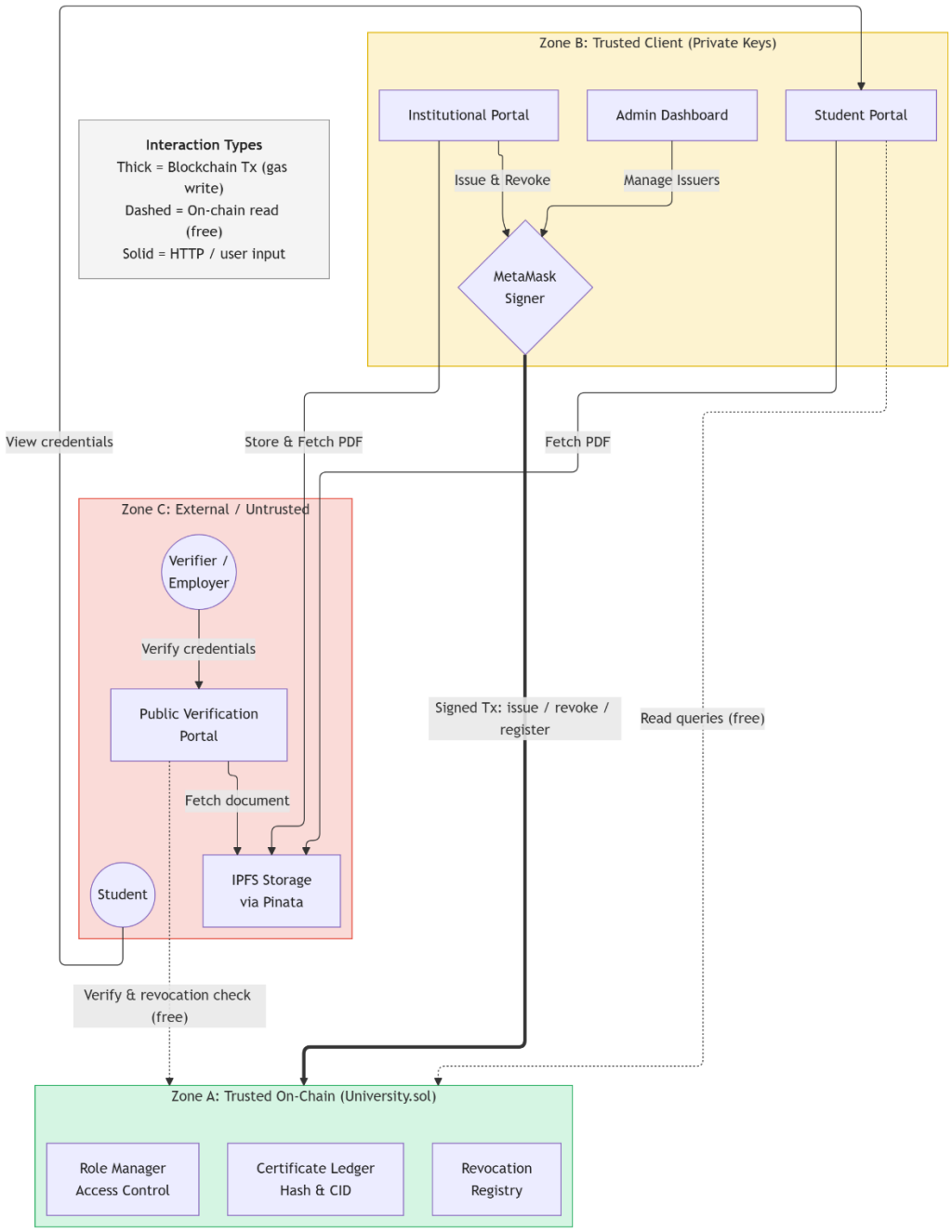
يتكوّن النظام من ثلاث طبقات رئيسية موزّعة على ثلاث مناطق ثقة (Trust Zones)، تُميَّز بحدود واضحة لإبراز أين تكمن المخاطر الأمنية وأين تُوقَّع المعاملات الحسّاسة. ويوضّح المخطط المبين في الصورة (7) البنية العامة والعلاقات بين هذه الطبقات، وتقسيم النظام إلى المناطق الآتية:

- **المنطقة (أ) — البيئة الموثوقة على السلسلة (Trusted On-Chain)**: وتضمّ العقد الذكي university.sol المنشور على شبكة الاختبار Sepolia. وهي مصدر الحقيقة الوحيد للنظام، وتحتوي منطقياً على ثلاث وحدات: سجل الشهادات (يخزّن البصمة ومعرّف المحتوى)، وسجل الإلغاء (يتتبّع حالة صلاحية كل شهادة)، ومدير الأدوار (يحكم صلاحيات الإصدار والإدارة). تُعدّ هذه المنطقة موثوقة لأن منطقها منشور علناً وغير قابل للتعديل بعد النشر.

- **المنطقة (ب) — بيئة العميل الموثوقة (Trusted Client)**: وتشمل واجهات التطبيق المبنية بإطار React (بوابة الطالب، بوابة المؤسسة، بوابة المدير)، إضافةً إلى محفظة MetaMask التي تحتفظ بالمفاتيح الخاصة وتتولّى توقيع المعاملات. سُمّيت موثوقة لأنها تعمل على جهاز المستخدم وتحت سيطرته المباشرة، ولأن المفاتيح الخاصة لا تغادرها أبداً.

- **المنطقة (ج) — البيئة الخارجية غير الموثوقة (External / Untrusted)**: وتضمّ الأطراف والخدمات التي تقع خارج سيطرة النظام، مثل جهات التحقق وأصحاب العمل، والطلبة كمستخدمين خارجيين، وبوابة التحقق العامة، وخدمة التخزين اللامركزي IPFS عبر مزوّد Pinata.

ويُلاحَظ أن كل معاملة تُغيّر حالة السلسلة (إصدار/إبطال/تسجيل) تمرّ إجبارياً عبر محفظة MetaMask لتوقيعها بالمفتاح الخاص، في حين أن عمليات القراءة والتحقق تُنفَّذ مباشرةً مقابل السلسلة دون رسوم، وهو ما يجعل التحقق من الشهادات مجانياً ومتاحاً للجميع.



صورة 7 – المعمارية العامة للنظام وتوزّع مناطق الثقة

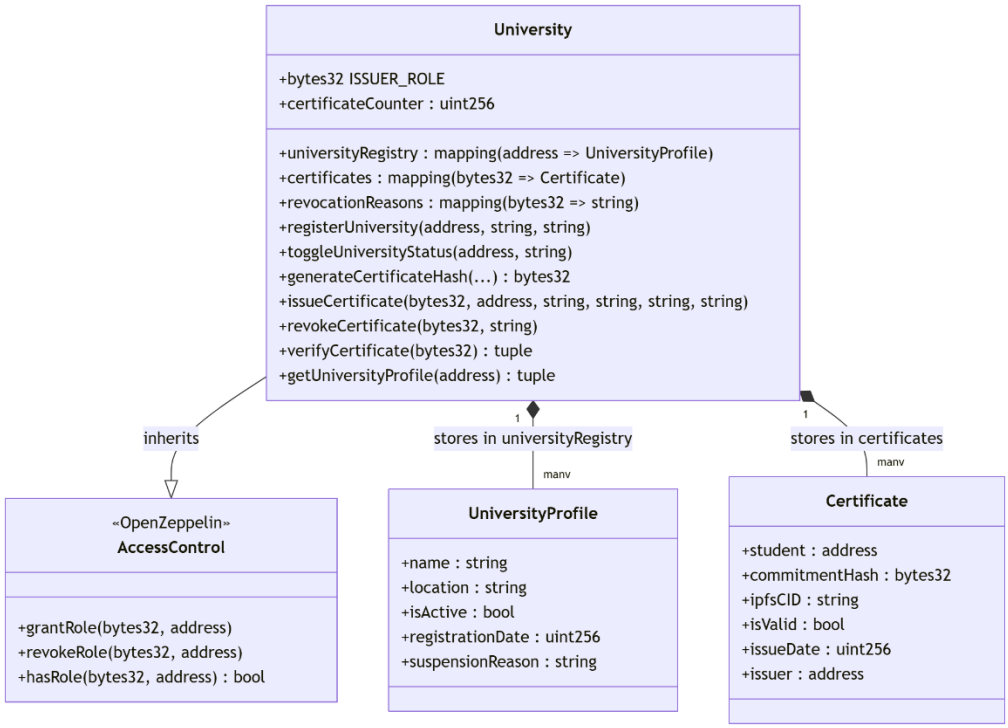
3.4 تصميم العقد الذكي (Smart Contract Design)

يمثّل العقد الذكي university.sol نواة النظام، وقد صُمّم اعتماداً على مكتبة OpenZeppelin AccessControl لإدارة الصلاحيات. وفيما يلي أبرز عناصره التصميمية.

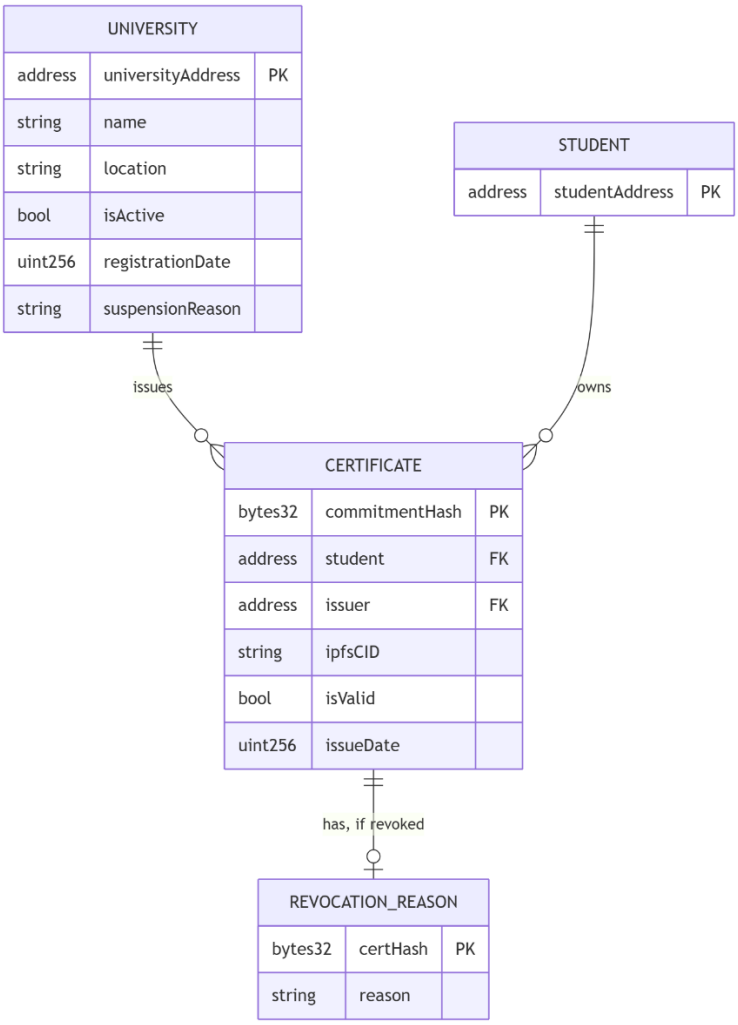
3.4.1 هياكل البيانات الرئيسية

- **بنية الشهادة Certificate**: تحتوي الحقول: عنوان الطالب studentAddress، وبصمة الالتزام commitmentHash، ومعرّف المحتوى ipfsCID، وحالة الصلاحية isValid، وتاريخ الإصدار issueDate، وعنوان المصدر issuerAddress.
- **سجل الجامعات universityRegistry**: خريطة (Mapping) من عنوان المؤسسة إلى بنية universityProfile (الاسم، الموقع، حالة التفعيل، تاريخ التسجيل، سبب التعليق).
- **سجل أسباب الإلغاء revocationReasons**: خريطة من بصمة الشهادة إلى نص سبب الإلغاء.
- **عدّاد الشهادات certificateCounter**: يُستخدم ضمن حساب البصمة لضمان تفرّدها.

ولتوضيح هذه البنية بصرياً، يعرض المخططان التاليان النموذج البرمجي للعقد الذكي من منظورين متكاملين: مخطط الأصناف (Class Diagram) الذي يبرز بنية العقد وتوابعه وعلاقته بمكتبة OpenZeppelin، ومخطط للعلاقات بين هياكل البيانات المخزَّنة على السلسلة، يُستخدم بديلاً عن مخطط الكيانات والعلاقات (ERD) التقليدي نظراً لغياب قاعدة بيانات علائقية في تصميم النظام.



صورة 8 – مخطط الأصناف (**Class Diagram**) للعقد الذكي **University.sol**



صورة 9– مخطط العلاقات بين هياكل البيانات المخزَّنة على السلسلة (بديل عن **ERD** نظراً لعدم وجود قاعدة بيانات علائقية)

يوضّح مخطط الأصناف (صورة 8) أن العقد University يرث من مكتبة AccessControl الخاصة بـ OpenZeppelin، ويحتفظ داخلياً بثلاث خرائط (mappings) تربط عناوين المحافظ وبصمات الشهادات ببنيتي UniversityProfile و Certificate، إلى جانب مجموعة التوابع المسؤولة عن التسجيل والإصدار والإلغاء والتحقق. أما مخطط العلاقات (صورة 9) فيبرز الروابط المنطقية بين هذه الهياكل: كل جامعة قد تُصدر عدداً من الشهادات، وكل شهادة ترتبط بطالب واحد وقد يُرفق بها سبب إلغاء عند تغيير حالتها. وخلافاً لمخطط الكيانات والعلاقات التقليدي المستخدم في قواعد البيانات العلائقية، لا تمثّل هذه الروابط جداول أو مفاتيح خارجية فعلية، بل علاقات منطقية بين خرائط مخزَّنة داخل حالة العقد الذكي على سلسلة الكتل، وهو ما يعكس طبيعة النظام اللامركزية القائمة على العقود الذكية بدلاً من قواعد البيانات المركزية.

3.4.2 نموذج التحكم بالوصول

اعتمد التصميم مكتبة OpenZeppelin AccessControl لتقييد التوابع الحسّاسة بالدورين DEFAULT_ADMIN_ROLE (الإدارة) و ISSUER_ROLE (الإصدار والإلغاء) تجسيداً لتوزّع الصلاحيات بين الفاعلين الموصوف في الفصل الثاني.

3.4.3 آلية الالتزام بالبصمة (Commitment Hash)

يحسب التابع generateCertificateHash بصمةً تشفيرية من مجموعة بيانات الشهادة (عنوان المصدِر، عنوان الطالب، الرقم الجامعي، الاختصاص، المعدّل، معرّف المحتوى، العدّاد). وعند الإصدار، يتحقّق العقد من تطابق البصمة المرسَلة مع البصمة المحسوبة داخلياً قبل التخزين، بما يمنع إدراج أي شهادة ببصمة غير مطابقة لمحتواها الفعلي.

3.4.4 التخزين اللامركزي (IPFS / Pinata)

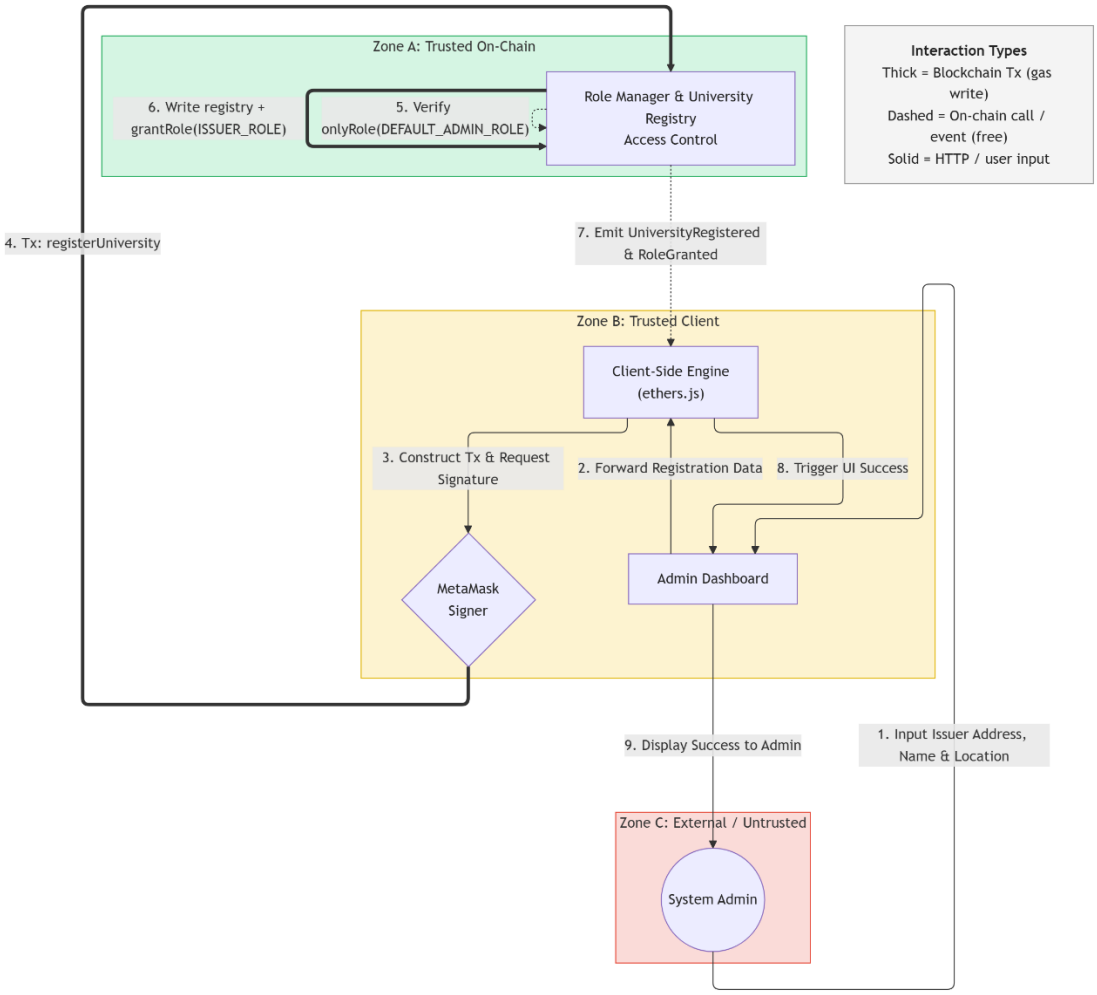
يُخزَّن ملف الشهادة الأصلي (PDF) على شبكة IPFS عبر مزوّد Pinata، ويُحفَظ معرّف المحتوى (CID) على السلسلة ضمن بنية الشهادة ليكون حلقة الوصل بين السجل والوثيقة الأصلية.

3.5 تصميم تدفّقات العمليات

تصف الفقرات الآتية التدفّق التفصيلي لكل عملية رئيسية في النظام. وتُقرأ جميع المخططات وفق دلالة الخطوط الموحّدة: الخط السميك يمثّل معاملةً على السلسلة تُغيّر الحالة وتكلّف غازاً، والخط المتقطّع يمثّل نداءً أو قراءةً على السلسلة دون رسوم، والخط العادي يمثّل تدفّق HTTP أو إدخالاً من المستخدم.

3.5.1 تدفّق تسجيل مؤسسة تعليمية ومنح دور الإصدار

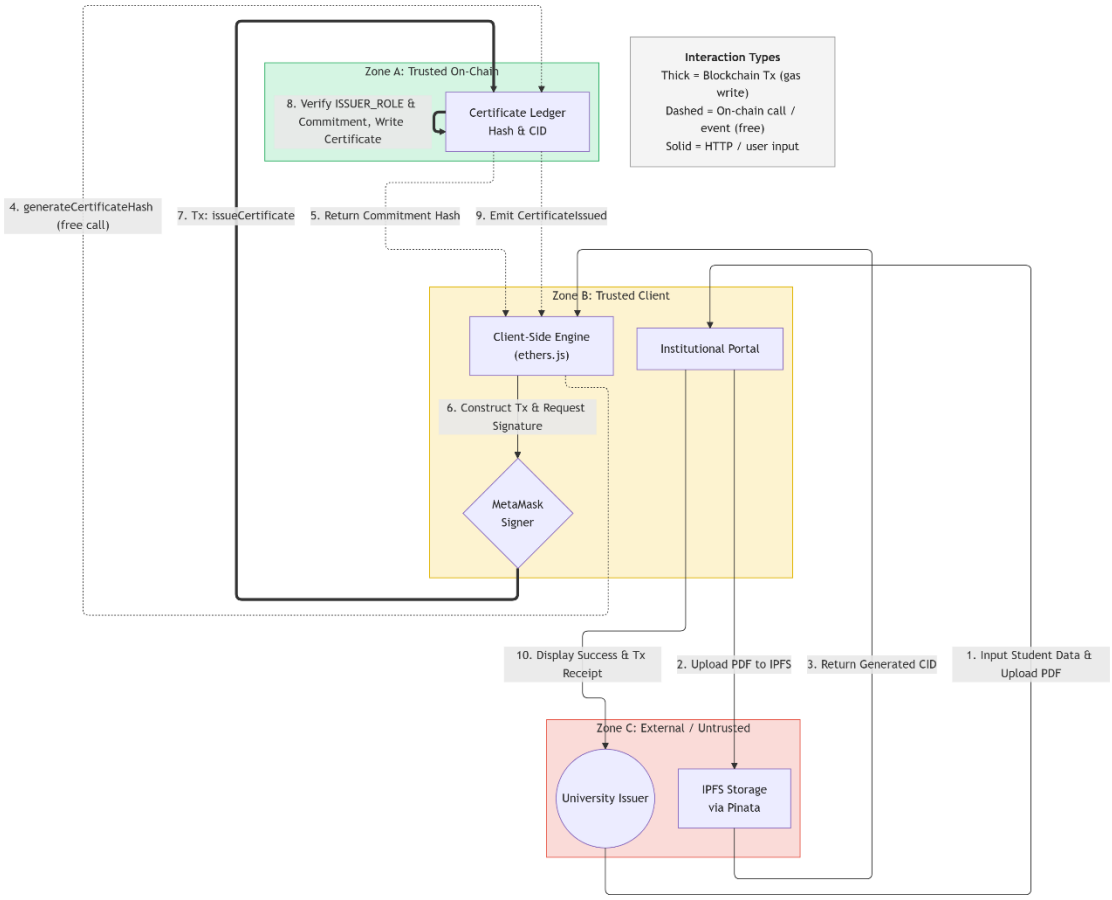
يوضّح المخطط التالي خطوات تسجيل مدير النظام لجامعة جديدة عبر التابع registerUniversity، بدءاً من إدخال بيانات الجامعة عبر لوحة الإدارة، مروراً بالتحقق من صلاحية DEFAULT_ADMIN_ROLE، وصولاً إلى منح دور الإصدار ISSUER_ROLE وإطلاق أحداث التسجيل.



صورة 10 – تدفّق تسجيل مؤسسة تعليمية ومنح دور الإصدار

3.5.2 تدفق إصدار شهادة

يوضّح المخطط في الصورة (11) تدفق إصدار الشهادة بدءاً من رفع ملف PDF إلى IPFS عبر Pinata، مروراً بحساب بصمة الالتزام عبر التابع generateCertificateHash، وصولاً إلى تنفيذ معاملة issueCertificate بعد التحقق المزدوج من الصلاحية وعدم تكرار البصمة.



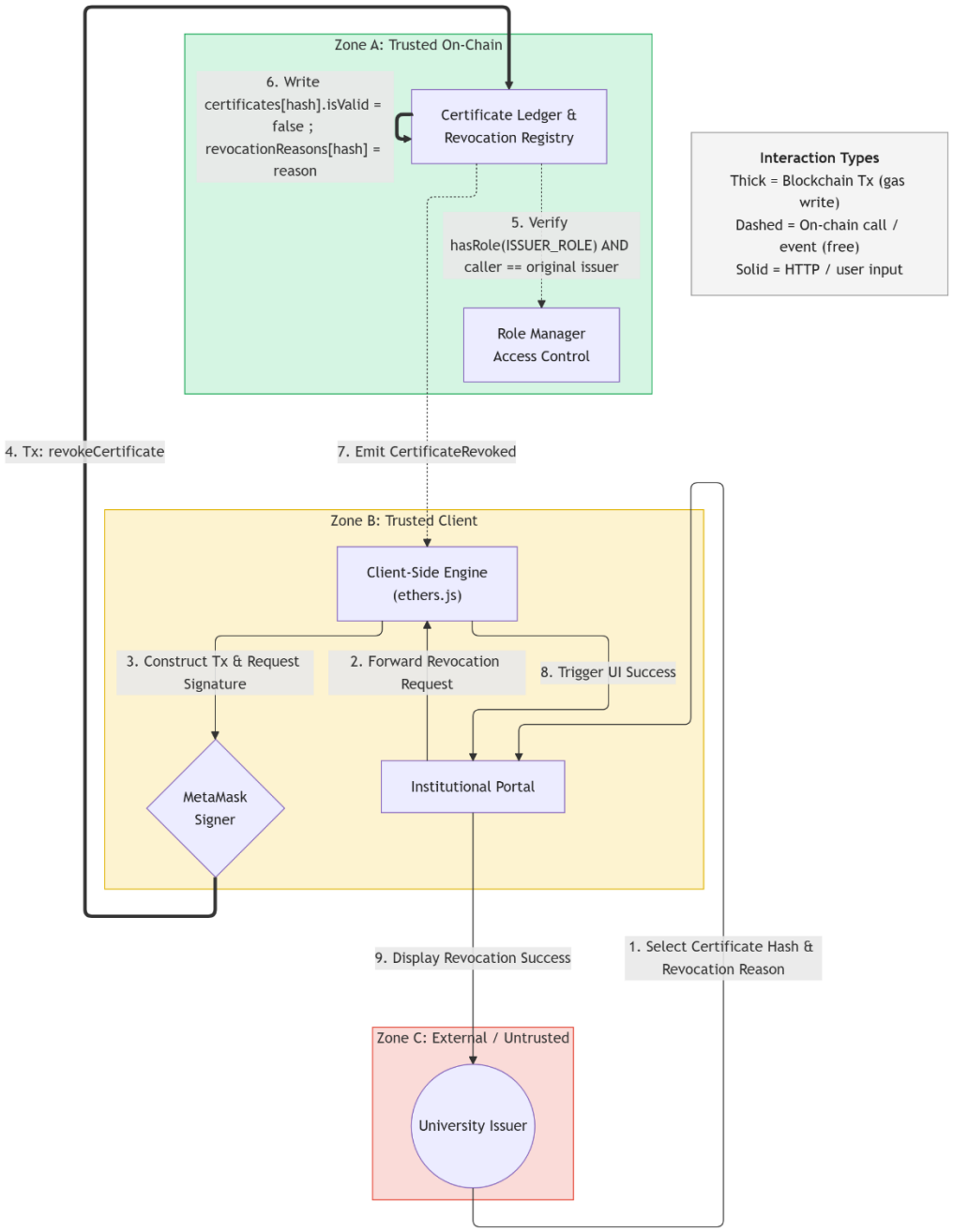
صورة 11 – تدفق إصدار شهادة

3.5.3 تدفق إلغاء الشهادة

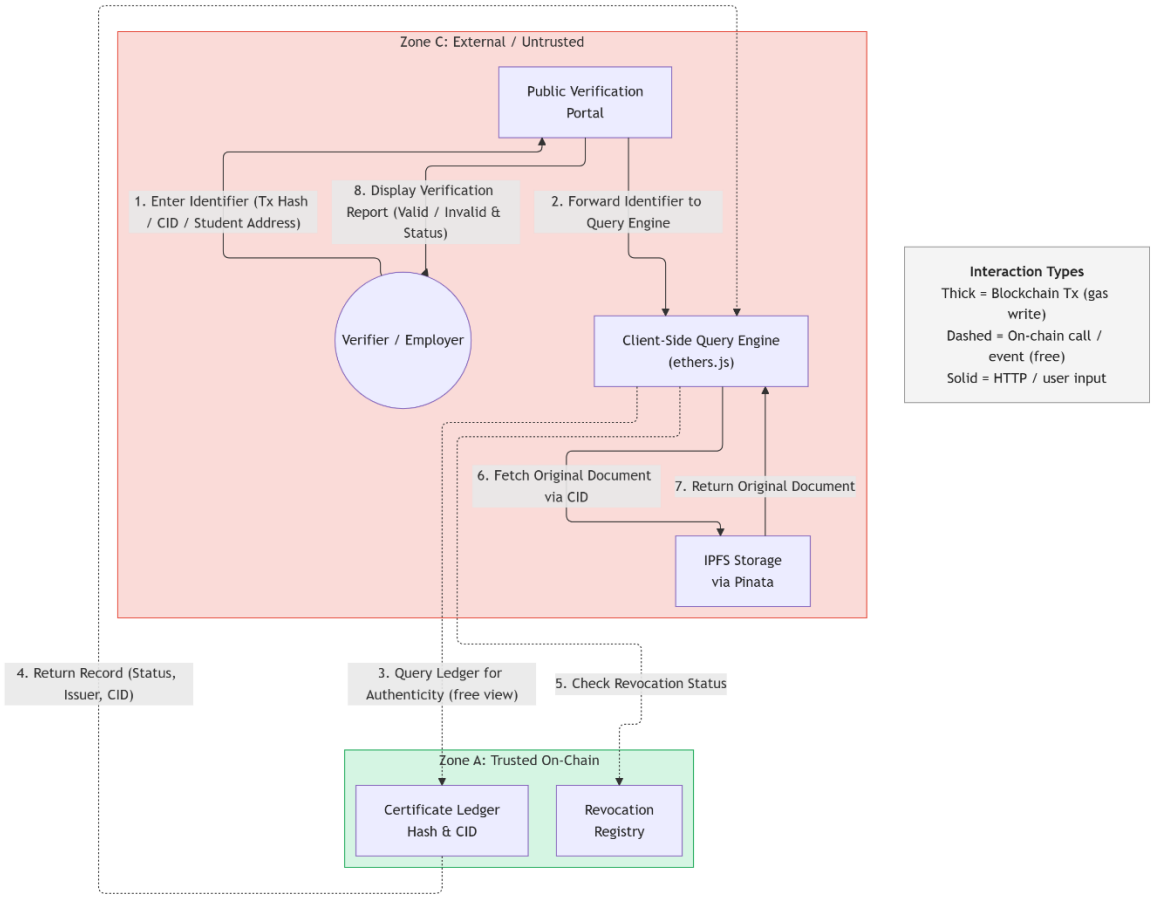
يوضّح المخطط (12) خطوات إلغاء الجامعة المصدرة لشهادة سبق إصدارها عبر التابع revokeCertificate، بعد التحقق من كون الجهة المستدعية هي المصدر الأصلي للشهادة، مع تخزين سبب الإلغاء دون حذف السجل.

3.5.4 تدفق التحقق من صحة الشهادة

يوضّح المخطط (13) آلية التحقق المجاني من صحة الشهادة عبر التابع verifyCertificate، حيث تسترجع جهة التحقق حالة الشهادة ومصدرها، ثم تجلب الوثيقة الأصلية من IPFS للمطابقة النهائية.



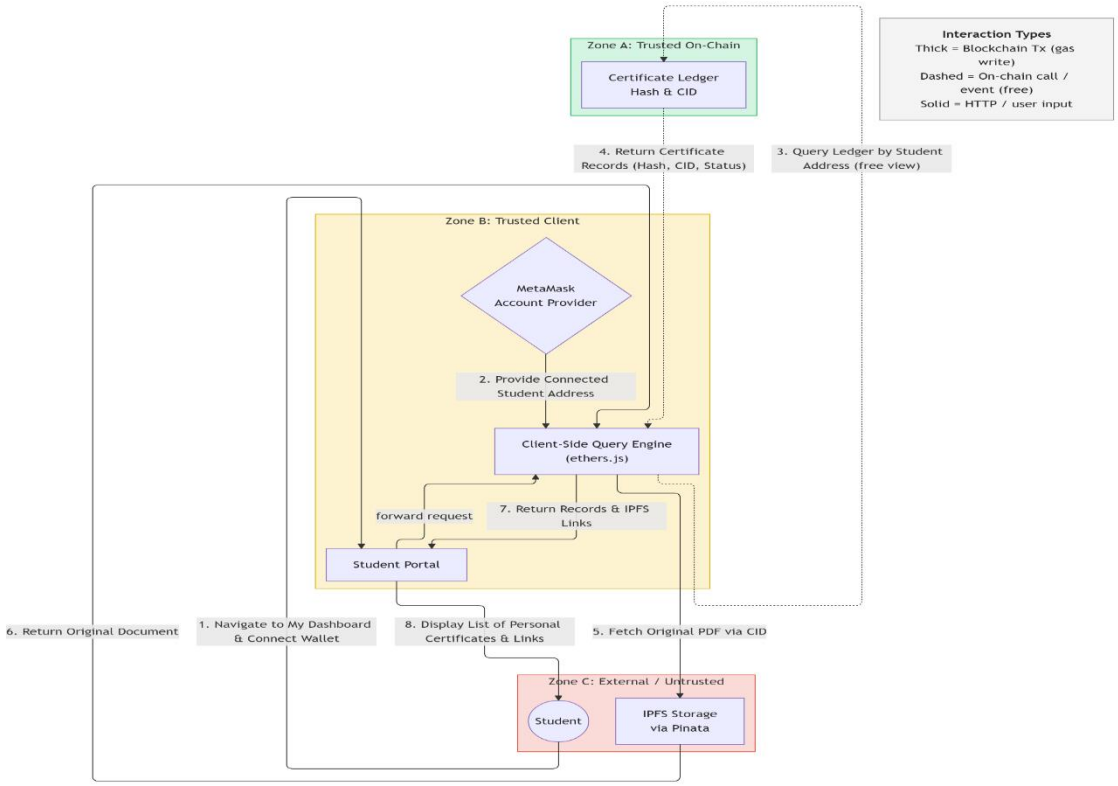
صورة 12 – تدفق إلغاء الشهادة



صورة 13 – تدفق التحقق من صحة الشهادة

3.5.5 تدفق استعراض الطالب لشهاداته

يوضّح المخطط (14) استرجاع الطالب لقائمة شهاداته الصادرة عبر ترشيح أحداث CertificateIssued المرتبطة بعنوان محفظته المتصلة، مع إتاحة الوصول إلى الملفات الأصلية عبر روابط IPFS.



صورة 14 – تدفق استعراض الطالب لشهاداته

3.6 اعتبارات الأمان والخصوصية

يجمع التصميم بين عدّة طبقات حماية:

- **سلامة المحتوى**: عبر بصمة الالتزام على السلسلة ومعرّف المحتوى المشتقّ من الملف على IPFS، بحيث يُكتشف أي تعديل فوراً.
- **الخصوصية**: وفق مبدأ الخصوصية بالتصميم المذكور في المبادئ التصميمية أعلاه.
- **التحكم بالوصول**: عبر نظام أدوار صارم يفصل بين الإدارة والإصدار، مع تحقّق مزدوج يمنع مؤسسةً من إلغاء شهادات غيرها.
- **حصر التوقيع**: بإبقاء المفاتيح الخاصة داخل المحفظة في بيئة العميل، وقصر تغيير الحالة على معاملات موقّعة.
- **قابلية التدقيق**: عبر الإبطال المنطقي الموضّح في تدفّق الإلغاء أعلاه، انسجاماً مع مبدأ الأثر التاريخي المطروح في الفصل الثاني.

الفصل الرابع

تنجيز النظام والاختبارات ومناقشة النتائج

يَعرض هذا الفصل كيفية تنجيز النظام مع تفصيل كل جزء من أجزائه والاختبارات التي تم تطبيقها للتأكد من تلبية النظام للمتطلبات.

4.1 مقدمة

يتناول هذا الفصل الترجمة العملية للتصميم المعماري الذي عُرض في الفصل السابق إلى نظام برمجي عامل. وبينما ركّز الفصل السابق على "ماهيّة" مكوّنات النظام وعلاقاتها، يركّز هذا الفصل على "كيفيّة" بنائها فعلياً على مستوى الكود البرمجي.

كما ويتناول هذا الفصل التحقّق من صحّة التنفيذ ومطابقته للمتطلّبات التي حُدّدت في الفصل الأول. وقد اعتُمدت استراتيجية اختبار على مستويين متكاملين: اختبارات وحدة آليّة تغطّي منطق العقد الذكي بوصفه جوهر النظام وطبقة الثقة فيه، واختبارات تكامل يدويّة تغطّي تدفّقات الواجهة الأربع من طرفٍ إلى طرف.

4.2 تنفيذ العقد الذكي (Smart Contract)

كُتب العقد University.sol بلغة Solidity (إصدار 0.8.28)، ويرث إدارة الصلاحيات من مكتبة OpenZeppelin AccessControl. ويعتمد نموذج الأدوار الموصّف في الفصل السابق (دور مدير النظام ودور الجهة المصدرة).

أهم هياكل البيانات المخزّنة على السلسلة:

- **UninversityProfile**: يحفظ اسم الجامعة وموقعها وحالتها (isActive) وتاريخ التسجيل وسبب الإيقاف إن وُجد.
- **Certificate**: يحفظ عنوان الطالب وبصمة الالتزام (commitmentHash) ومعرّف IPFS (ipfsCID) وحالة الصلاحية (isValid) وتاريخ الإصدار والجهة المصدرة).
- **ترابطات (mappings)**: universityRegistr و certificates و revocationReasons، إضافةً إلى عدّاد certificateCounter.

أهم التوابع المنَفَّذة:

- **registerUniversity**: تسجيل جامعة جديدة ومنحها صلاحية ISSUER_ROLE.
- **toggleUniversityStatus**: إيقاف جامعة أو إعادة تفعيلها مع منح/سحب الصلاحية تبعاً للحالة.
- **generateCertificateHash**: تابع pure يحسب بصمة الالتزام باستخدام keccak256.
- **issueCertificate**: إصدار شهادة بعد التحقّق من فعالية الجهة المصدرة ومطابقة البصمة وعدم وجودها مسبقاً، ثمّ إطلاق الحدث CertificateIssued.
- **revokeCertificate**: إلغاء منطقي للشهادة (تعيين isValid=false) مع تسجيل السبب، بعد التحقّق من أنّ المنفّذ هو الجهة المصدرة الأصلية، وإطلاق الحدث CertificateRevoked.
- **VerifyCertificate** و **getUniversityProfile**: توابع قراءة (view) تُرجع حالة الشهادة وبياناتها أو ملف الجامعة دون أي تكلفة.

وفيما يلي مقتطف تابع الإصدار issueCertificate الذي يجمع منطق التحقق والتثبيت:

```
function issueCertificate(
    bytes32 _certHash, address _student, string memory _studentId,
    string memory _major, string memory _gpa, string memory
_ipfsCID
) public onlyRole(ISSUER_ROLE) {
    require(universityRegistry[msg.sender].isActive, "Issuer is
suspended in registry");
    bytes32 expectedHash = generateCertificateHash(
        msg.sender, _student, _studentId, _major, _gpa, _ipfsCID,
certificateCounter
    );
    require(_certHash == expectedHash, "Invalid certificate
hash");
    require(certificates[_certHash].issueDate == 0, "Certificate
hash already exists");
    certificates[_certHash] = Certificate({
        student: _student, commitmentHash: _certHash, ipfsCID:
_ipfsCID,
        isValid: true, issueDate: block.timestamp, issuer:
msg.sender
    });
    certificateCounter += 1;
    emit CertificateIssued(_certHash, _student, _ipfsCID);
}
```

وتوضح الخوارزمية التالية خطوات حساب بصمة الالتزام وإصدار الشهادة:

```
//Commitment Calculation and Certificate Issuance
Input:  issuer, student, studentId, major, gpa, pdfFile
Output: commitmentHash anchored on-chain

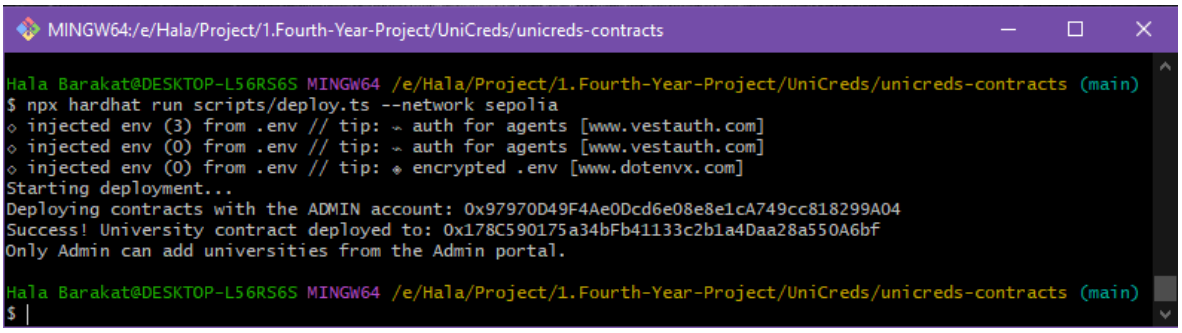
1:  cid        <- IPFS.addAndPin(pdfFile)              // off-chain content addressing
2:  counter    <- contract.certificateCounter()
3:  commitment <- Keccak256( issuer, student, studentId,
                             major, gpa, cid, counter(
4:  require( hasRole(ISSUER_ROLE, issuer) )            // access authorization
5:  require( certificates[commitment].issueDate == 0 ) // prevent duplication
6:   contract.issueCertificate(commitment, student, studentId, major, gpa, cid)
7:  return commitment
```

4.3 نشر العقد على الشبكة (Deployment)

جرى نشر العقد عبر سكربت deploy.ts بإطار Hardhat إلى شبكة الاختبار Sepolia، من خلال بوّابة عقدة RPC من خدمة Infura (المعرّفة في المتغيّر SEPOLIA_RPC_URL). وقبل النشر جرى تمويل محفظة النشر بعملة ETH تجريبية من Google Cloud Sepolia Faucet لتغطية رسوم الغاز. وبعد نجاح النشر يُوضع عنوان العقد في متغيّر البيئة NEXT_PUBLIC_CONTRACT_ADDRESS لتستخدمه الواجهة. كما يدعم السكربت تسجيل جامعة أولية اختيارياً عبر متغيّرات البيئة. وفيما يلي مقتطف من سكربت النشر deploy.ts:

```
const University = await
hre.ethers.getContractFactory("University");
const contract = await University.deploy();
await contract.waitForDeployment();
const contractAddress = await contract.getAddress();
console.log(`University contract deployed to: ${contractAddress}`);
```

وتوضّح الصورة التالية مخرجات نشر العقد على شبكة Sepolia:



صورة 15 – مخرجات نشر العقد على شبكة **Sepolia**

4.4 تنفيذ طبقة التخزين اللامركزي (IPFS / Pinata)

جرى رفع ملفّ الشهادة (PDF) إلى شبكة IPFS عبر مسار واجهة برمجية يعمل على الخادم (Server-Side API Route) هو /api/pinata. والحكمة من جعله خادمياً أنّ مفتاح Pinata السرّي (PINATA_JWT) يبقى على الخادم ولا يتسرّب إلى المتصفّح. ويُعيد المسار معرّف المحتوى (CID) الذي يُثبّت لاحقاً على السلسلة.

ويتم تنفيذ ذلك كما هو موضح في الخوارزمية التالية:

```
export const runtime = "nodejs";
export async function POST(request: Request) {
  const jwt = process.env.PINATA_JWT;            // secret stays
server-side
  const incoming = await request.formData();
  const file = incoming.get("file");
  const payload = new FormData();
  payload.append("file", file, file.name);
  const response = await fetch(
    "https://api.pinata.cloud/pinning/pinFileToIPFS",
    { method: "POST", headers: { Authorization: `Bearer ${jwt}` },
body: payload }
  );
  const data = await response.json();
  return NextResponse.json({ cid: data.IpfsHash });
}
```

4.5 طبقة الربط مع السلسلة في الواجهة

جرى تجميع كلّ التعامل مع العقد في وحدة مركزية هي lib/university-contract.ts، وهي توفّر:

- مُزوّد قراءة (JsonRpcProvider) مع إعادة محاولة تلقائية (retry/backoff) لمعالجة انقطاع مزوّد RPC المؤقّت.

- عقد كتابة (getWriteContract) يُشتقّ من محفظة MetaMask (BrowserProvider + signer) مع التحقّق من تطابق الحساب النشط.
- توابع مساعدة للتمييز بين أنواع المدخلات (isHash وisCid) وتوجيه الاستعلام fetchCredentialByQuery.
- قراءة شـــهادات الطالب fetchStudentCredentials عبر ترشـــيح الحدث CertificateIssued الخاص بعنوان الطالب.

4.6 تنفيذ واجهات المستخدم

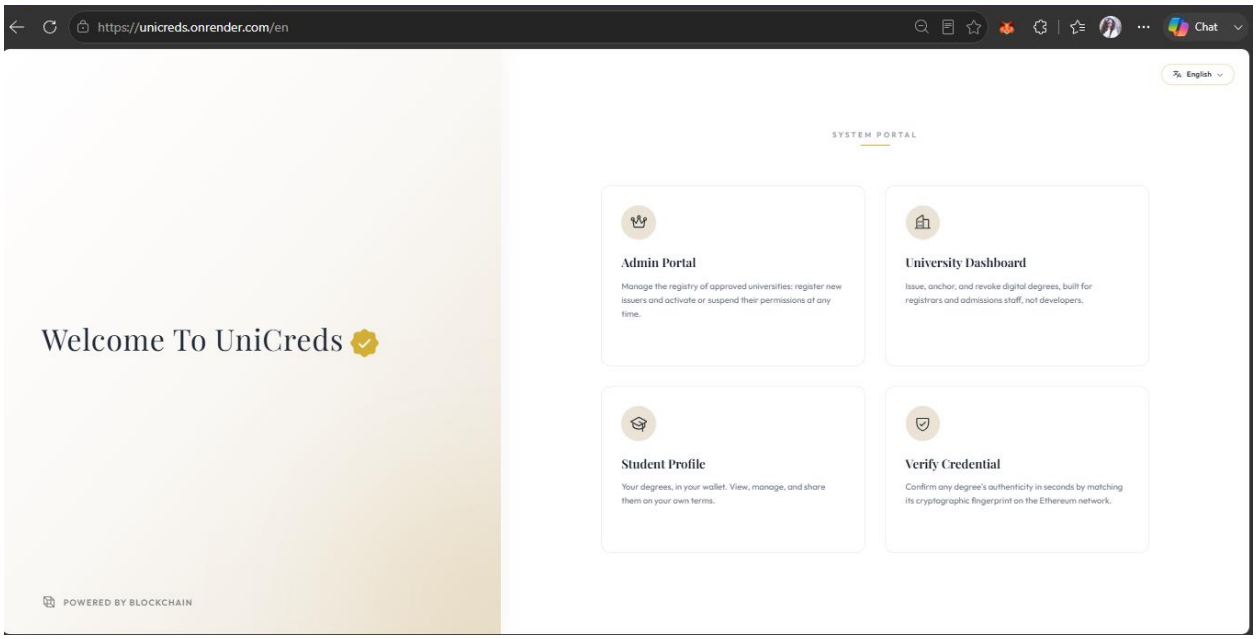
بُنيت الواجهات باستخدام App Router في Next.js، مع بنية [locale] لدعم اللغتين العربية والإنجليزية (i18n) عبر طبقة middleware. وتنقسم الواجهة إلى أربع بوّابات تقابل الأدوار المعرّفة في الفصل الثاني:

الصفحة الرئيسية: تعرض بوّابات الدخول إلى الأدوار.

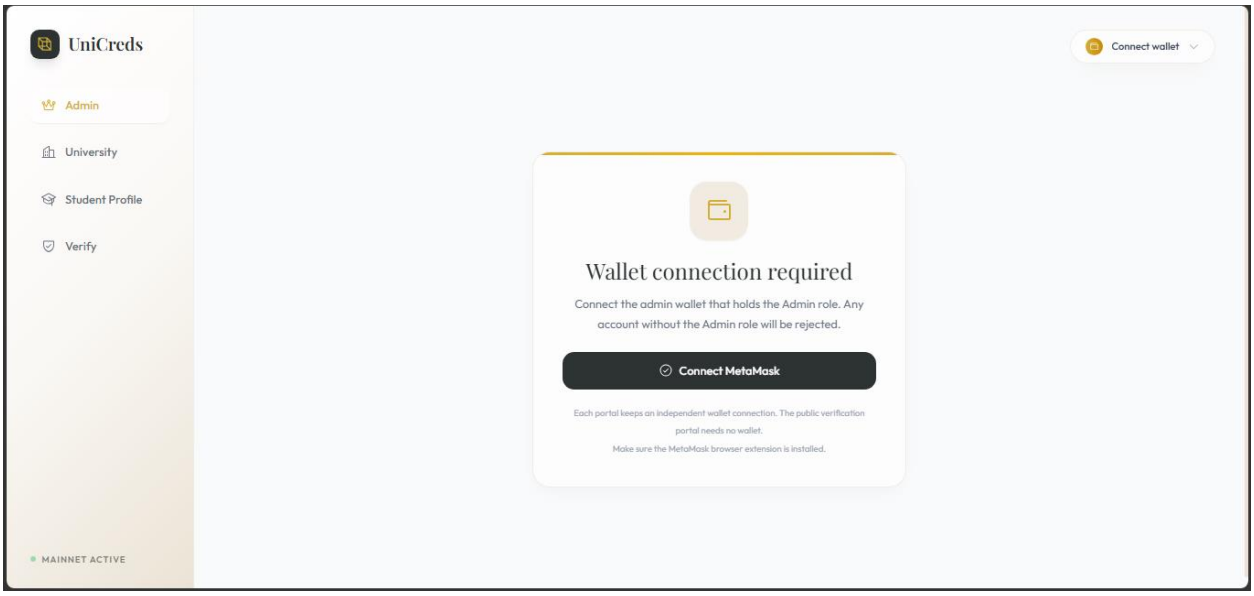
- بوّابة مدير النظام (AdminDashboard): تسجيل الجامعات وإدارة حالتها (إيقاف/تفعيل).
- بوّابة الجامعة (IssueForm وRecordsTable): إصدار الشهادات وإلغاؤها واستعراض سجلّها.
- بوّابة الطالب (StudentDashboard وCredentialCard): عرض شهاداته الخاصة.
- بوّابة التحقّق (VerifyPanel): تحقّق علني من الشهادة بإدخال البصمة أو الـ CID أو عنوان الطالب.

ويُمثّل نموذج الإصـــدار (IssueForm) التدفّق الجوهري للنظام، إذ يجمع بين رفع الملف والحســـاب المجّاني للبصـــمة ثمّ المعاملة المكلّفة (وفق التمييز بين القراءة المجّانية والكتابة المكلّفة الموضّح في الفصل السابق).

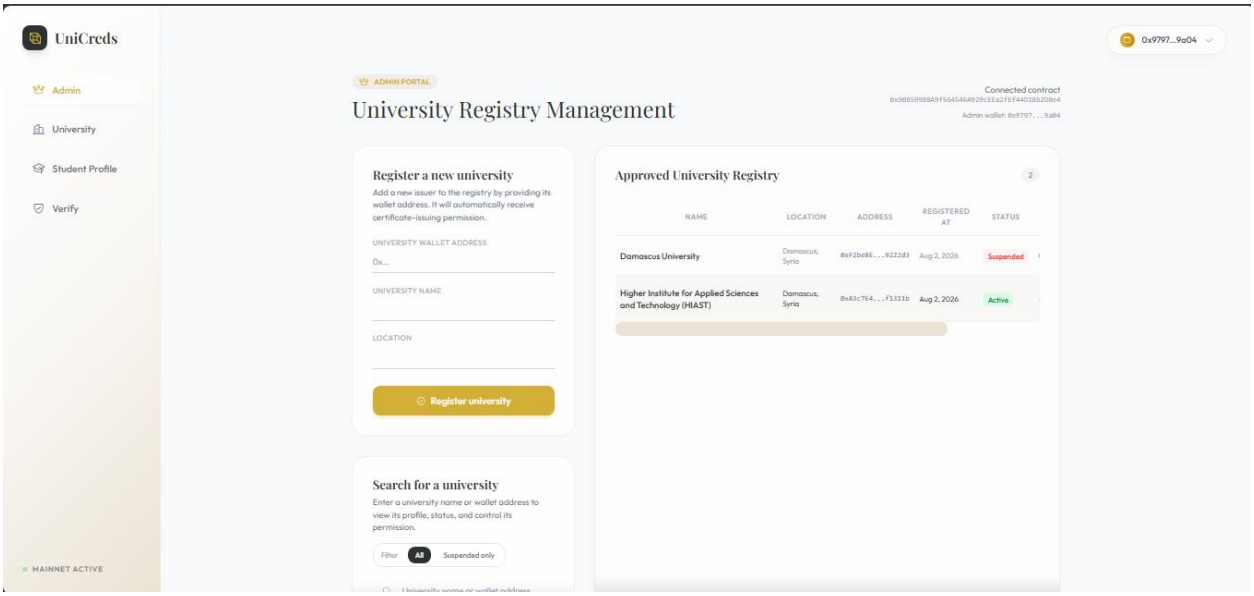
وتوضـح الصـور التالية الواجهة الرئيسـية للنظام، والبوابات الأربع المذكورة أعلاه، مع واجهات طلب الاتصـال بالمحفظة ونافذة MetaMask:



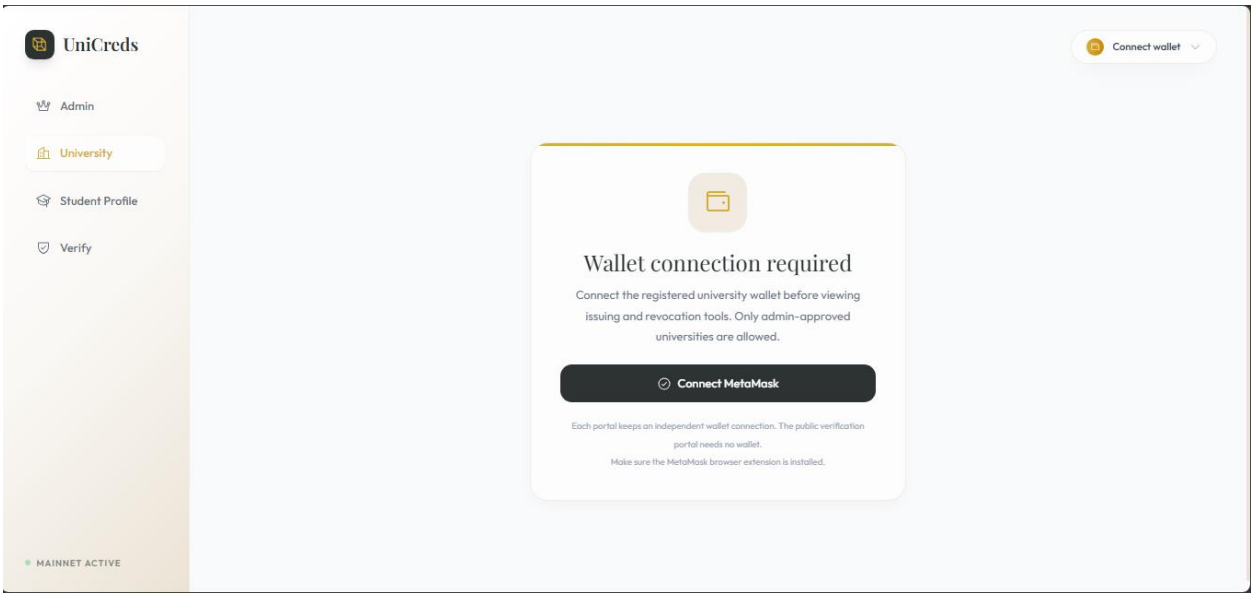
صورة 16 – الصفحة الرئيسية لبوابات النظام



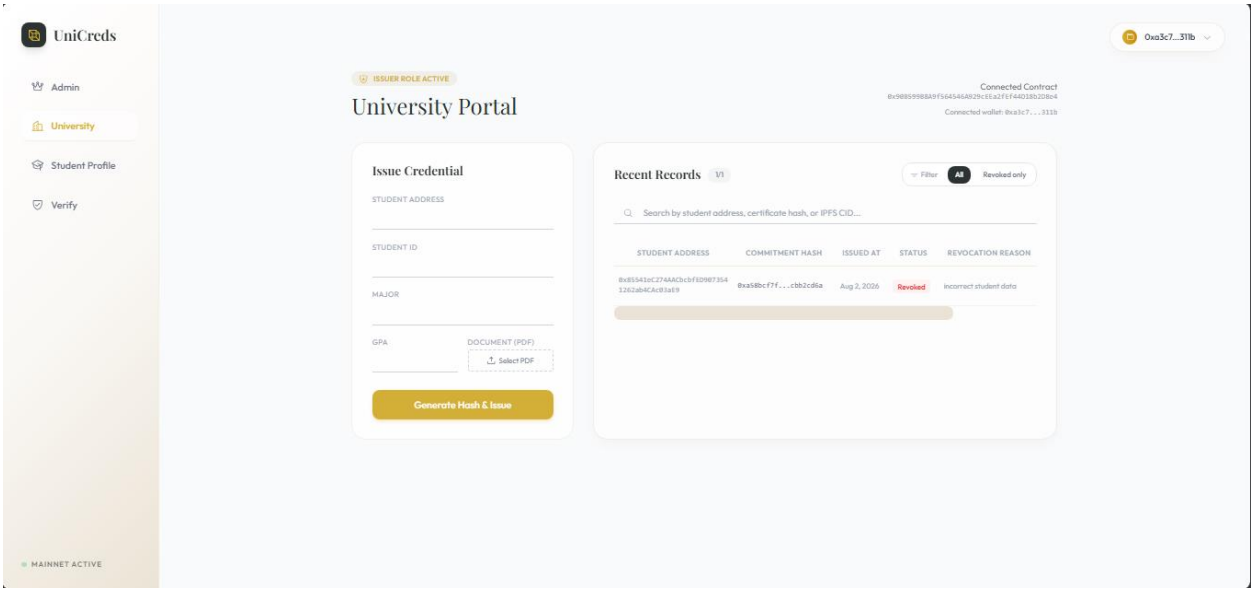
صورة 17 – بوابة مدير النظام – واجهة الاتصال بالمحفظة



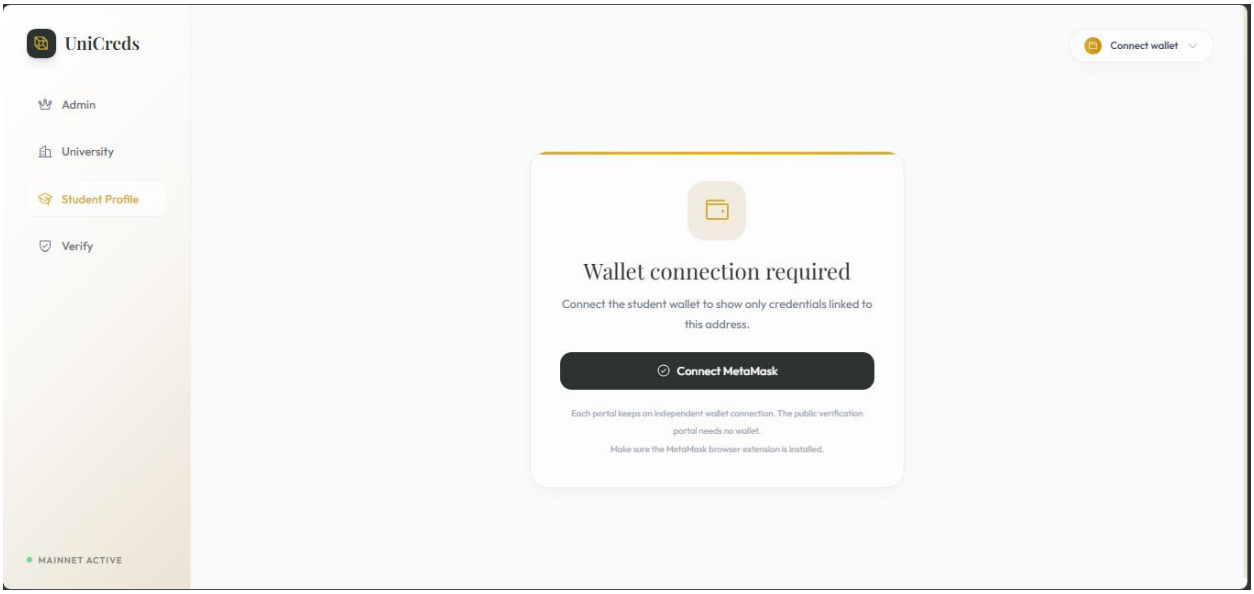
صورة 18 – بوابة مدير النظام – تسجيل وإدارة الجامعات



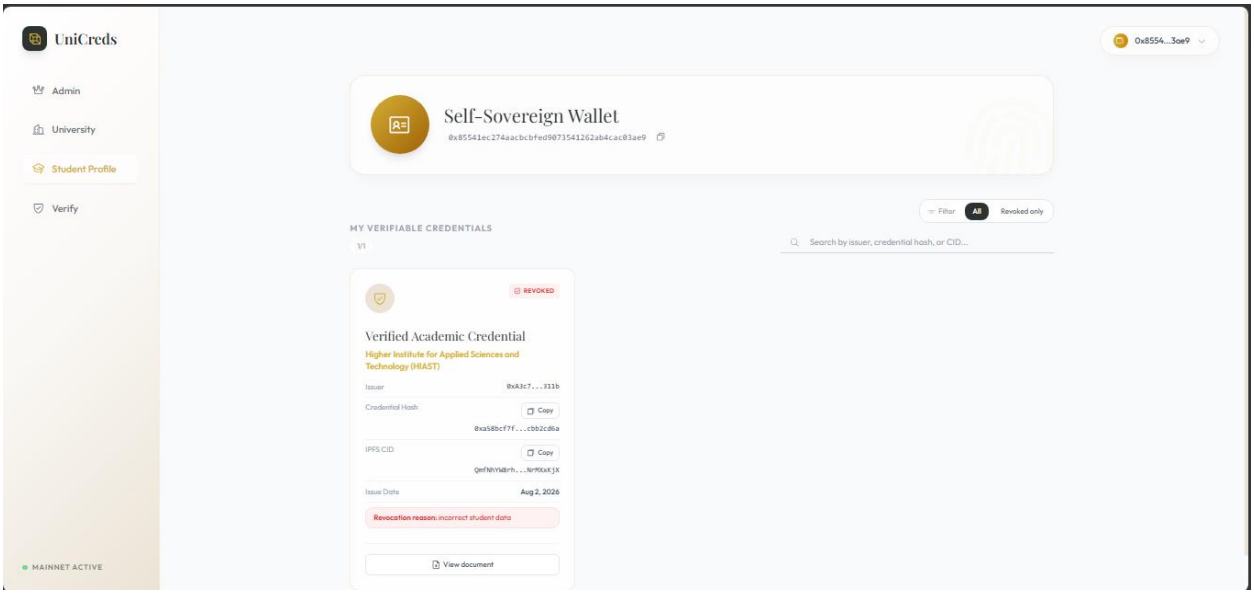
صورة 19 – بوابة الجامعة – واجهة الاتصال بالمحفظة



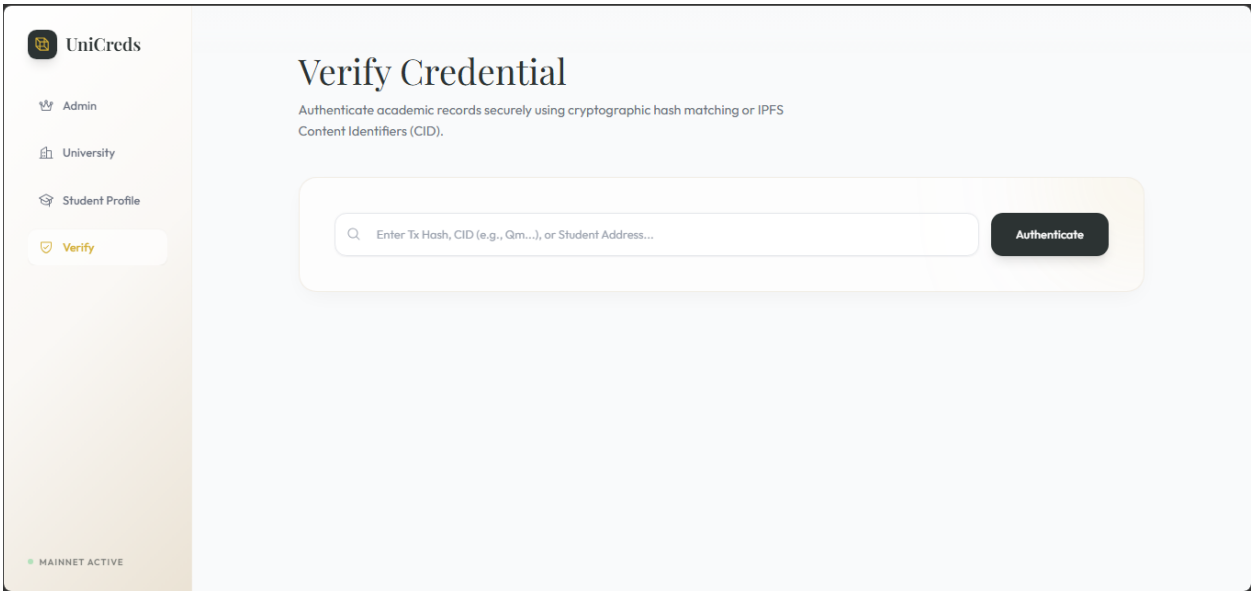
صورة 20 – بوابة الجامعة – نموذج إصدار الشهادة



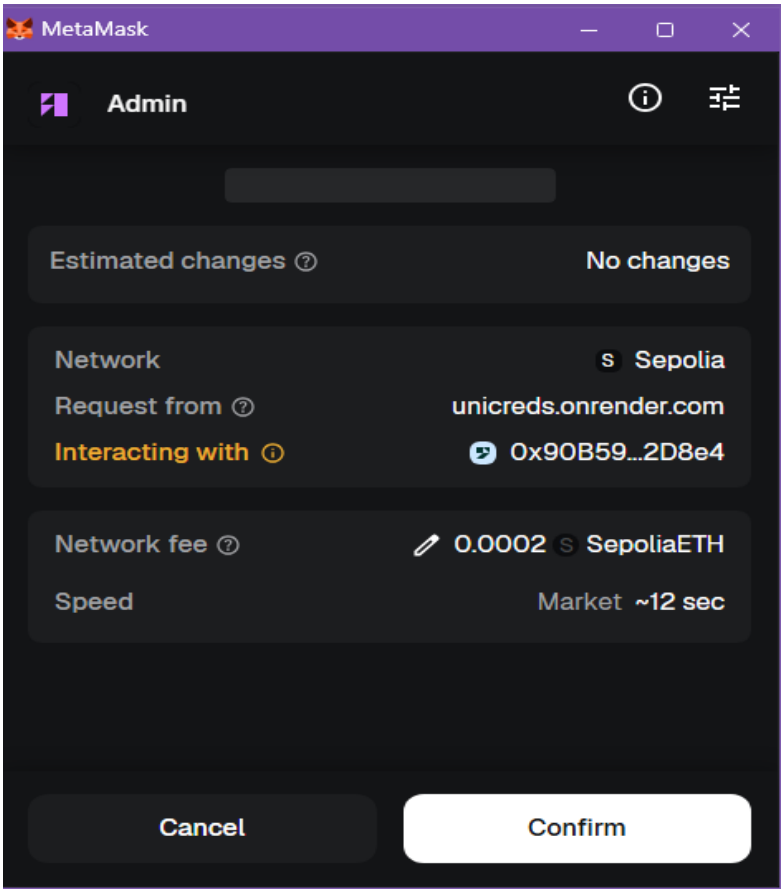
صورة 21 – بوابة الطالب – واجهة الاتصال بالمحفظة



صورة 22 – بوابة الطالب – بطاقات الشهادات



صورة 23 – بوابة التحقق



صورة 24 – نافذة **MetaMask** لتأكيد معاملة تنشيط جامعة موقوفة مسبقاً

4.7 معالجة الأخطاء وتجربة المستخدم

جرى تحويل رسائل التحقّق (require) القادمة من العقد — مثل "Not original issuer" أو "Certificate hash already exists" — إلى رسائل مفهومة ثنائية اللغة عبر التابع parseContractError، مع إشعارات نجاح (toasts) ومعالجة لانقطاع مزوّد RPC. ويضمن ذلك تجربة مستخدم واضحة ومتسقة مع مسارات الأخطاء الموصّفة في حالات الاستخدام (الفصل الثاني).

4.8 مطابقة المتطلبات مع التنفيذ

لضمان تغطية جميع المتطلبات المحددة في الفصل الأول فعلياً ضمن التنفيذ، يعرض الجدولان التاليان مطابقة مباشرة بين كل متطلب وظيفي وغير وظيفي والجزء البرمجي المسؤول عن تحقيقه.

الجدول 1 – المتطلبات الوظيفية والأجزاء البرمجية المسؤولة عن تحقيقها

#	المتطلب الوظيفي	الجزء البرمجي المنفِّذ
1	إدارة المؤسسات المعتمدة.	التابع registerUniversity، بوابة مدير النظام.
2	تفعيل/إيقاف المؤسسات وإدارة حالتها.	التابع toggleUniversityStatus، بوابة مدير النظام (جدول إدارة الجامعات).
3	التحكم في الوصول المبني على الأدوار.	وراثة العقد من OpenZeppelin AccessControl، الدوران DEFAULT_ADMIN_ROLE و ISSUER_ROLE.
4	منح/سحب صلاحية الإصدار تلقائياً حسب حالة المؤسسة.	استدعاء grantRole/revokeRole داخل toggleUniversityStatus.
5	تخزين الشهادات خارج السلسلة عبر IPFS.	مسار الواجهة الخلفية api/pinata، خطوة الرفع في IssueForm.
6	ربط الشهادة بالوثيقة الأصلية.	الحقل ipfsCID ضمن بنية Certificate ومعاملات issueCertificate.
7	توليد الالتزام/البصمة التشفيرية.	التابع generateCertificateHash.
8	إصدار الشهادات.	التابع issueCertificate، نموذج IssueForm.
9	إلغاء الشهادات من المصدر الأصلي فقط.	شرط require في revokeCertificate للتحقق من هوية المصدر.
10	حفظ سبب الإلغاء.	الخريطة revocationReasons، جدول السجلات RecordsTable.
11	التحقق العام من الشهادات.	التابع verifyCertificate، بوابة التحقق VerifyPanel.
12	توثيق العمليات عبر الأحداث.	الأحداث UniversityRegistered و CertificateIssued و CertificateRevoked.

13	واجهة ويب متعددة البوابات حسب الدور.	البوابات الأربع: الأدمن، الجامعة، الطالب، التحقق.

الجدول 2 – المتطلبات غير الوظيفية وكيفية تحققها ضمن المشروع

#	المتطلب غير الوظيفي	كيفية تحقيقه
1	الأمان	فرض onlyRole على جميع التوابع الحساسة، مراجعة يدوية وآلية (القسم 9.5).
2	الخصوصية	تخزين البصمة و CID فقط على السلسلة دون بيانات الطالب الفعلية.
3	نزاهة البيانات وعدم القابلية للتلاعب	الالتزام التشفيري عبر abi.encode و Keccak256.
4	كفاءة التخزين واستهلاك الموارد	تخزين الملفات خارج السلسلة عبر IPFS، الاكتفاء بالحقول الأساسية على السلسلة.
5	قابلية التوسع	تكلفة غاز ثابتة لكل عملية إصدار، مؤكَّدة عبر اختبار الحمل (القسم 9.6).
6	الموثوقية	اجتياز جميع حالات اختبار الوحدة (11/11، القسم 9.3).
7	التوافرية والاستمرارية	توابع القراءة العامة (verifyCertificate، getUniversityProfile) متاحة دون رسوم وفي أي وقت.
8	قابلية الاستخدام	بوابات مبسّطة مخصصة لكل دور، بواجهة ثنائية اللغة (عربي/إنكليزي).
9	قابلية الصيانة	الفصل بين العقد وسكربتات التشغيل والواجهة.
10	قابلية الاختبار	حزمة اختبارات آلية عبر Hardhat، واختبارات تكامل يدوية (القسمان 9.3 و9.4).

11	التوافقية وقابلية التكامل	التكامل عبر معايير قياسية: MetaMask و ethers.js و IPFS/Pinata.

4.9 أدوات التنفيذ

يعرذ الجدول التالي الأدوات والتقنيات وبيئات العمل التي استُخدمت في بناء نظام UniCreds وتنفيذه ونشره، موزّعةً على محاور المشروع الأساسية: تطوير العقد الذكي، وبيئة سلسلة الكتل والنشر، وطبقة التخزين اللامركزي، وتطوير الواجهة الأمامية، إضافةً إلى الأدوات المساعدة وبيئة الاستضافة. وقد رُوعي في اختيار هذه الأدوات أن تكون مفتوحة المصدر وواسعة الاعتماد في منظومة إيثيريوم، بما يضمن الموثوقية وقابلية الصيانة والتوافق مع المعايير القياسية.

الجدول 11 – الأدوات والتقنيات المستخدمة في تنفيذ نظام **UniCreds** موزَّعة على طبقات النظام

الطبقة	الأدوات والتقنيات
العرض والتفاعل (Frontend)	Next.js 15، React 19، TypeScript، Tailwind CSS، ethers.js 6.x، MetaMask
السلسلة والعقد الذكي (On-Chain)	Solidity 0.8.28، Hardhat، مكتبة OpenZeppelin AccessControl، شبكة اختبار Sepolia
التخزين اللامركزي (Off-Chain)	IPFS، خدمة التثبيت Pinata
التطوير والنشر	Git وGitHub، Node.js، Chai، TypeChain، منصة الاستضافة Render

4.10 بيئة الاختبار وأدواته

نُفّذت اختبارات العقد الذكي باستخدام إطار Hardhat مع مكتبة التأكيدات Chai، وأداة hardhat-network-helpers التي توفّر آليّة loadFixture لإعادة تهيئة حالة العقد قبل كل اختبار على شبكة محليّة سريعة ومعزولة. ويُهيّئ تابع التثبيت الأوّلي أربعة حسابات اختبارية بأدوار مختلفة: مدير النظام (admin)، والجهة المصدِرة (issuer)، والطالب (student)، وحساب غير مخوّل (otherAccount)، لتغطية مسارات الصلاحيات كافّة.

4.11 اختبار العقد الذكي (Unit Testing)

تتكوّن حزمة الاختبار من إحدى عشرة حالة اختبار موزّعة على مجموعتين، تغطّيان التحكّم في الوصول والسجلّ من جهة، وعمليّات الشهادة وبصمة الالتزام من جهة أخرى. ويلخّص الجدولان التاليان هذه الحالات وأهدافها ونتائجها المتوقّعة.

الجدول 3 – مجموعة الاختبارات الأولى – التحكّم في الوصول والسجلّ (**Access Control & Registry**)

حالة الاختبار	الهدف	النتيجة المتوقعة
تعيين ناشر العقد مديراً للنظام.	التأكد من أن حساب الناشر يملك دور مدير النظام DEFAULT_ADMIN_ROLE.	القيمة true.
تسجيل جامعة ومنح الدور.	قيام مدير النظام بتسجيل جامعة ومنحها صلاحية الإصدار ISSUER_ROLE.	إطلاق الحدث UniversityRegistered ومنح الدور بنجاح.
طلب سبب الإيقاف ومسحه.	رفض إيقاف جامعة دون سبب، ثمّ الإيقاف بسبب وسحب الدور، ثمّ إعادة التفعيل ومسح السبب.	فشل "Suspension reason required"، ثمّ إطلاق UniversityStatusToggled وضبط الدور والحالة والسبب بدقّة.
منع غير المخوّلين من الإصدار.	محاولة حساب لا يملك ISSUER_ROLE إصدار شهادة.	الرفض بالخطأ AccessControlUnauthorizedAccount.

وتوضح الخوارزمية التالية آلية الفحص المزدوج للنزاهة أثناء عملية التحقق:

```
//Dual-Integrity Check during Verification
Input:  certHash
Output: verification report

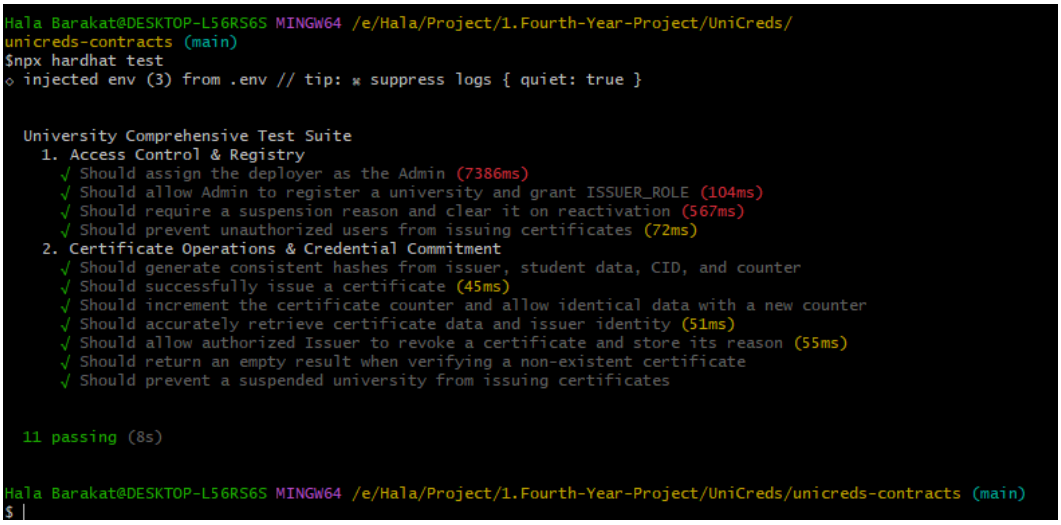
1:  cert <- certificates[certHash]                  // gas-free view
call
2:  if cert.issueDate == 0 then
3:       return "NOT FOUND - never issued"            // existence
check
4:  if cert.isValid == false then
5:      return "REVOKED", revocationReasons[certHash] // revocation
check
6:   document <- IPFS.get(cert.ipfsCID)                 // retrieve
original document
7:  return { valid: true, issuer, student, issueDate, document }
```

الجدول 4 – مجموعة الاختبارات الثانية – عمليّات الشهادة وبصمة الالتزام (**Certificate Operations & Credential Commitment**)

حالة الاختبار	الهدف	النتيجة المتوقّعة
اتّساق حساب البصمة.	مطابقة بصمة العقد للبصمة المحسوبة محلياً بـ solidityPackedKeccak256.	تطابق البصمتين تماماً.
إصدار شهادة بنجاح.	إصدار شهادة من جامعة مُفعَّلة.	إطلاق الحدث CertificateIssued بالوسائط الصحيحة.
زيادة العدّاد وتفرّد البصمة.	إصدار شهادتين ببيانات متطابقة مع عدّاد مختلف.	زيادة certificateCounter واختلاف البصمة الثانية عن الأولى.
استرجاع بيانات الشهادة.	قراءة الشهادة عبر verifyCertificate بعد إصدارها.	إرجاع (صالحة، عنوان الطالب، تاريخ > 0، عنوان المصدِر، CID) بدقّة.
الإلغاء وتخزين السبب.	إلغاء المصدِر المخوّل لشهادة مع سبب.	إطلاق حدث CertificateRevoked، وضبط isValid=false، وتخزين السبب.
شهادة غير موجودة.	التحقّق من بصمة غير مُسجَّلة.	إرجاع (false، عنوان صفري، 0، سلسلة فارغة).

منع جامعة مُوقوفة من الإصدار.	محاولة جامعة أُوقِفت إصدار شهادة بعد سحب دورها.	الرفض بالخطأ .AccessControlUnauthorizedAccount

وتوضح الصورة التالية نجاح الاختبارات الإحدى عشرة المذكورة أعلاه جميعها.

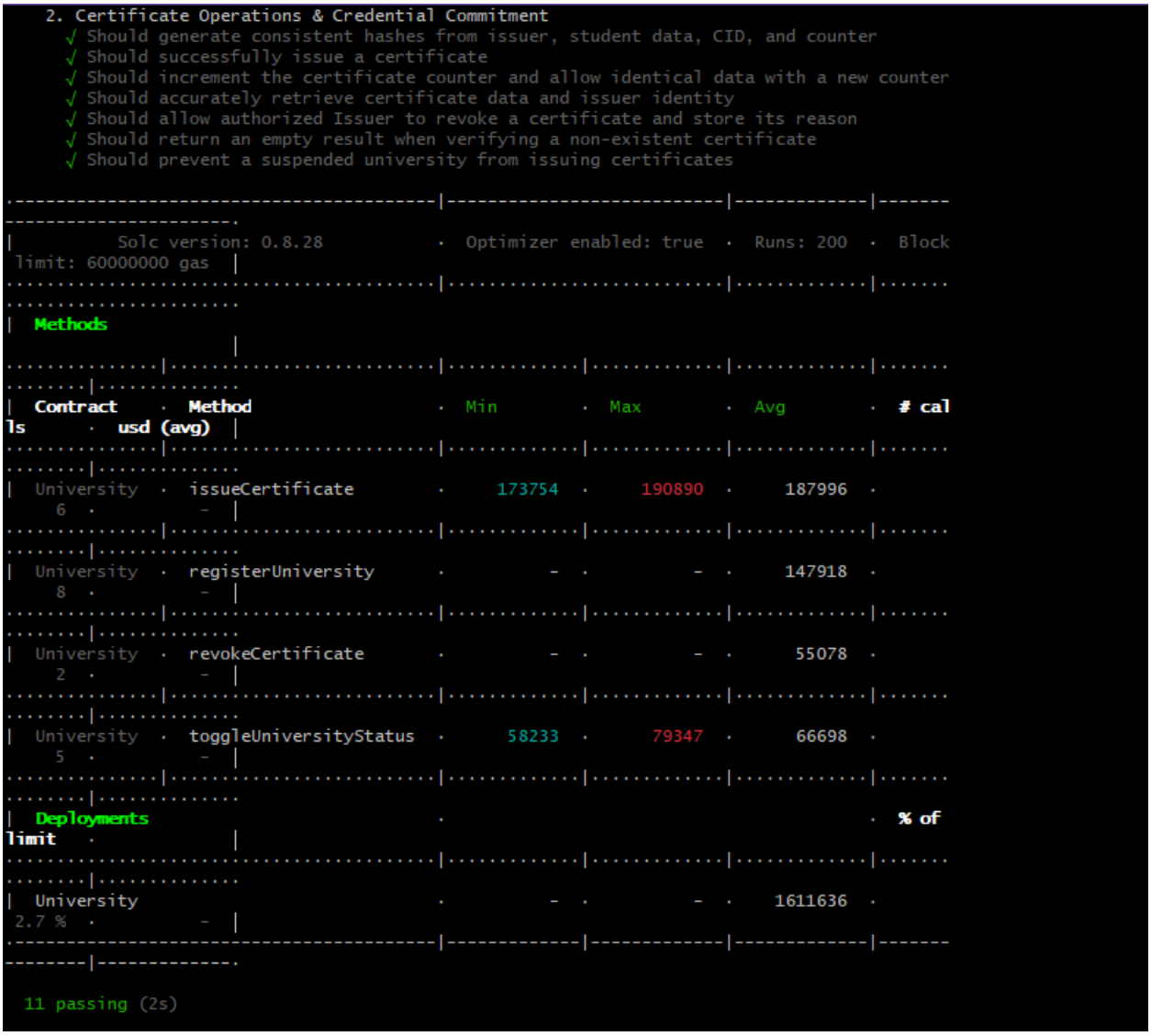


صورة 25 – نتيجة تشغيل حزمة اختبار العقد الذكي (اجتياز الحالات الإحدى عشرة).

كما وتوضح المخرجات الموضحة في الصورة (26) تقرير استهلاك الغاز لتوابع العقد.

4.12 اختبار التكامل والواجهة

بما أنّ الاختبارات الآليّة تغطّي منطق العقد، جرى التحقّق من تدفّقات الواجهة من طرفٍ إلى طرف يدويّاً عبر السيناريوهات التالية، بالربط مع حالات الاستخدام المعرّفة في الفصل الثاني.



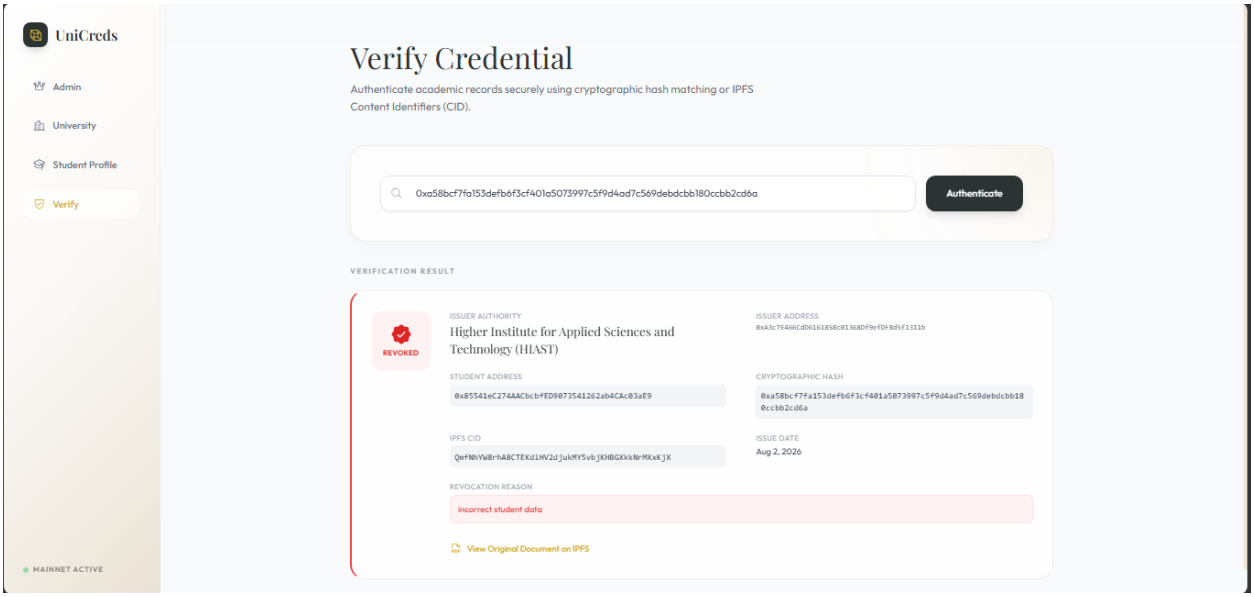
صورة 26 – تقرير استهلاك الغاز لتوابع العقد

الجدول 5 – مجموعة اختبارات التكامل والواجهة

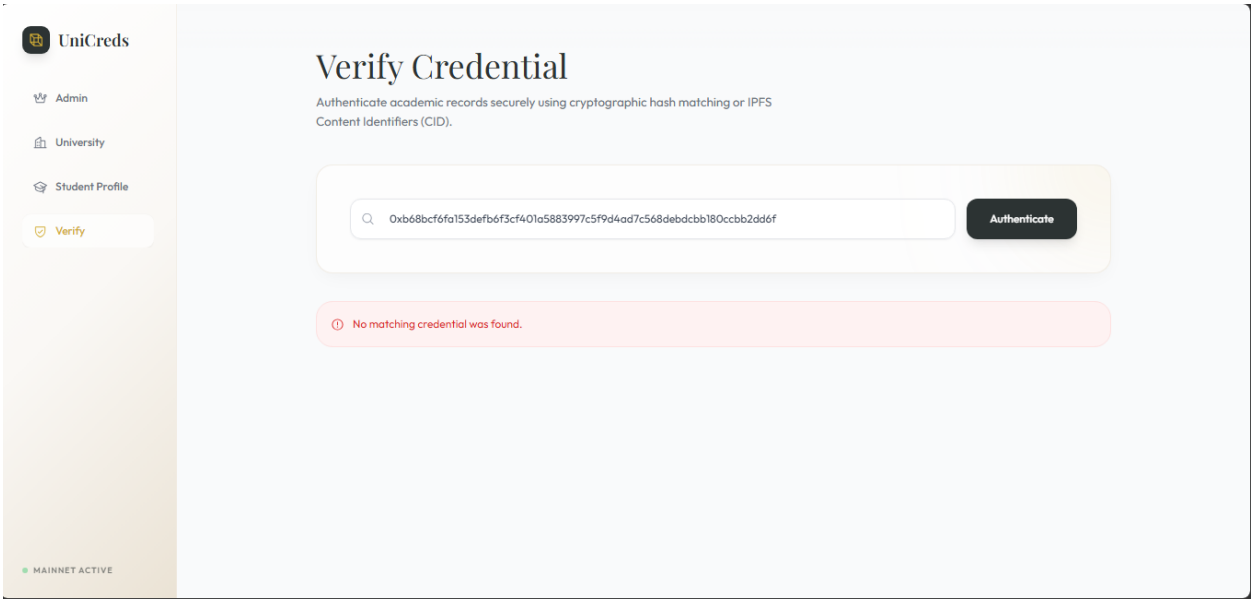
السيناريو	النتيجة المتوقّعة
تسجيل جامعة (مدير النظام).	ظهور الجامعة ضمن القائمة ومنحها صلاحية الإصدار.
إصدار شهادة (الجامعة).	رسالة نجاح وظهور الشهادة في سجلّ الجامعة.

التحقّق العلني.	ظهور النتيجة "صالحة" مع بيانات الشهادة ورابط IPFS.
عرض شهادات الطالب.	ظهور بطاقات الشهادات الخاصّة بالطالب.
الإلغاء (الجامعة المصدِرة).	تحوّل الحالة إلى "ملغاة" وظهور السبب عند التحقّق.
حالات الخطأ.	ظهور رسالة خطأ واضحة ومترجمة عبر parseContractError.
تعدّد اللغة.	تبدّل النصوص واتجاه الواجهة تبعاً للّغة.

وفيما يلي عيّنةٌ من نتائج اختبارات الواجهة اليدوية (الصورتين 29 و 30) التي أُجريت من طرفٍ إلى طرف، تُبيّن استجابة النظام في حالتين تمثيليّتين: عرض نتيجة التحقّق من شهادةٍ مُلغاة، ومعالجة محاولة التحقّق من بصمةٍ غير موجودة، بما يؤكّد مطابقة السلوك الفعلي للنظام لِما هو متوقَّع في كلٍّ من مسار النجاح ومسارات الخطأ.



صورة 27 – نتيجة التحقّق من شهادة مُلغاة تُظهر الحالة "ملغاة" وسبب إلغائها



صورة 28 – رسالة الخطأ عند التحقّق من شهادة غير مُسجَّلة

4.13 اختبار الأمان (Security Testing)

للتأكد من سلامة العقد الذكي University.sol من الناحية الأمنية، أُجري تقييم على مستويين متكاملين: مراجعة يدوية وفق قائمة تحقق تغطي أنماط الثغرات الشائعة في العقود الذكية، وتحليل ساكن آلي باستخدام أداة Slither.

4.13.1 المراجعة اليدوية

لا يحتوي العقد على أي استدعاءات خارجية أو تحويلات لعملة Ether، وبالتالي فهو غير معرّض لهجمات إعادة الدخول (Reentrancy). كما تخضع جميع التوابع التي تُغيّر الحالة للتحقق من الصلاحيات عبر المعدِّل onlyRole، وقد جرى التأكد من ذلك عملياً ضمن حزمة اختبارات الوحدة (الجدول 12). ولأن العقد مكتوب بإصدار Solidity أحدث من 0.8، فإن فيض الأعداد (Overflow/Underflow) محميٌ تلقائياً من قبل المترجم (Compiler)، دون حاجة لفحص إضافي. كما لا يعتمد العقد على tx.origin في أي عملية تحقق، ولا يحتوي على أي حلقات تكرارية على مصفوفات، مما ينفي احتمال هجمات استنزاف الغاز الناتجة عن التكرار غير المحدود.

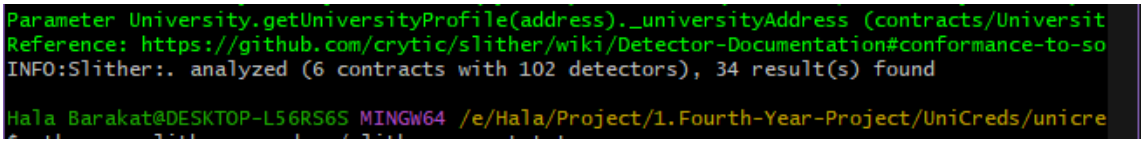
فيما يخص توليد بصمة الاعتماد، يستخدم التابع generateCertificateHash الدالة abi.encode لتجميع مدخلات الشهادة قبل حساب البصمة عبر Keccak256. اختيار abi.encode تحديداً يضمن إضافة حشو وفواصل أطوال ثابتة بين كل معامل، مما يمنع إمكانية أن تُنتج مجموعتا بيانات مختلفتان البصمة نفسها (Hash Collision) عند تجميع عدة قيم نصية

متتالية. وقد تحقّقت حزمة اختبارات الوحدة من ثبات هذا السلوك، إذ اجتازت جميع الحالات الإحدى عشرة (11 passing) بما فيها حالة اختبار توليد بصمة متسقة لمجموعة معطاة من المدخلات.

وبما أن التابع issueCertificate يعتمد على قيمة certificateCounter المقروءة مسبقاً عبر استدعاء منفصل لـ generateCertificateHash قبل إرسال معاملة الإصدار، ففي حال إرسال الجهة المصدرة نفسها معاملتين متقاربتين زمنياً، قد تعتمد كلتاهما على القيمة ذاتها للعدّاد، فترفض الشبكة إحداهما بسبب عدم تطابق البصمة. هذا السلوك لا يشكّل ثغرة قابلة للاستغلال من طرف خارجي، إذ يقتصر أثره على معاملات الجهة المصدرة لنفسها، ويمكن تفاديه عملياً بإرسال المعاملات بالتتابع بدل التزامن.

4.13.2 التحليل الآلي

جرى تشغيل أداة Slither على كامل مستودع العقد، والتي تفحص الكود عبر 102 كاشفاً آلياً مخصصاً لأنماط الثغرات المعروفة في Solidity. أظهر التشغيل 34 نتيجة ضمن خمس فئات، روجعت جميعها وتبيّن أنها لا تمثّل ثغرات فعلية: فئتا incorrect-equality و boolean-equal ناتجتان عن مقارنة حقول أولية مثل issueDate == 0 للتحقق من وجود سجل، وهو نمط آمن شائع في Solidity. وفئة timestamp تشير إلى المقارنات ذاتها دون أن يعتمد العقد فعلياً على block.timestamp لأي غرض حساس كالعشوائية أو التوقيت. أما فئتا pragma و solc-version فناتجتان بالكامل عن مكتبات OpenZeppelin المستوردة ضمن مجلد الاعتماديات (node_modules) وليس من كود المشروع نفسه. وأخيراً، فئة naming-convention هي ملاحظة أسلوبية بحتة تتعلق باستخدام الشرطة السفلية كبادئة لمعاملات التوابع، وهي ممارسة متعمّدة لتمييز معاملات التوابع عن متغيرات الحالة داخل العقد.



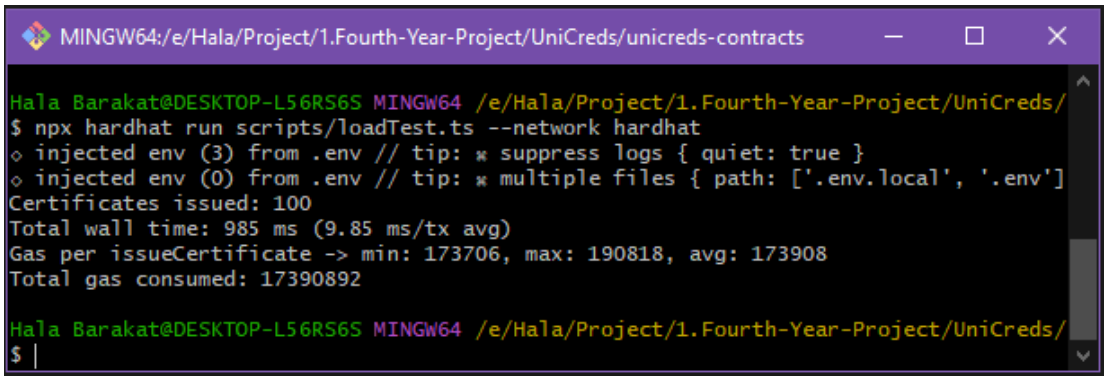
صورة 29 – نتيجة تشغيل أداة التحليل الساكن **Slither** على العقد الذكي **University.sol**

4.14 اختبار الحمل (Load Testing)

تم تنفيذ اختبار حمل محلي عبر شبكة Hardhat باستخدام سكربت loadTest.ts، بإصدار 100 شهادة متتالية من الجهة المصدرة نفسها، بهدف قياس سلوك استهلاك الغاز وزمن التنفيذ مع تراكم عدد السجلات داخل العقد. أظهرت النتائج استهلاكاً إجمالياً للغاز قدره 17,390,892 وحدة عبر 100 عملية إصدار، بمتوسط 173,908 وحدة لكل عملية (بين حد أدنى 173,706 وحد أقصى 190,818)، أي بتفاوت طفيف لا يتجاوز 10% ناتج عن اختلاف طول السلاسل النصية المدخلة (رقم الطالب ومعرّف IPFS) بين التكرارات المتتابعة.

والأهم من قيمة الاستهلاك ذاتها هو أن تكلفة الغاز لكل عملية لم تتزايد مع تراكم عدد الشهادات المخزَّنة سابقاً في العقد، ما يؤكد تكلفة ثابتة ((O(1)) لكل عملية إصدار بغض النظر عن حجم السجل التاريخي، وهو ما يدعم عملياً متطلب قابلية التوسع (البند 5 في القسم 1.4).

أما زمن التنفيذ الكلي فقد بلغ 985 مللي ثانية لتنفيذ 100 معاملة، أي بمعدل 9.85 مللي ثانية للمعاملة الواحدة. ويعكس هذا الرقم زمن التنفيذ الحسابي داخل بيئة محاكاة محلية خالية من زمن الشبكة الحقيقي، وليس معدل معالجة الشبكة الفعلي على Sepolia، حيث يخضع زمن التأكيد الفعلي هناك لزمن الكتلة في الشبكة العامة (نحو 12 ثانية) وليس لأداء العقد نفسه. وتوضح الصورة التالية نتائج الاختبار.



صورة 30 – نتائج اختبار الحمل (إصدار 100 شهادة على شبكة **Hardhat** المحلية)

4.15 تقييم النتائج ومطابقة المتطلبات

تؤكّد نتائج الاختبارات تحقيق المتطلّبات الوظيفيّة الأساسيّة المذكورة في الفصل الأول: إصدار الشهادات والتحقّق منها وإلغاؤها، وإدارة صلاحيات الجهات المصدِرة عبر التحكّم بالأدوار. كما تدعم النتائج المتطلّبات غير الوظيفيّة: اللامركزيّة والشفافيّة عبر تثبيت السجلّ على السلسلة، وخصوصيّة بيانات الطالب عبر اعتماد بصمة الالتزام بدل تخزين البيانات صراحةً (وفق الفصل السابق)، وكفاءة الكلفة عبر اعتماد عمليّات القراءة المجّانيّة في التحقّق والاستعراض. وبذلك يكون النظام قد اجتاز التحقّق الوظيفي وغطّى مسارات الأخطاء المتوقّعة.